

CONFIDENCIAL

Auditoria defensiva de segurança

GestorFinance

Data: 2026-09-14 · **Commit:** 903f44660ca2 (main)

Modo: STATIC_ONLY + testes locais seguros em localhost com conta demo e dados canário (criados e removidos). Produção EXCLUÍDA.

Runtime: Node v24.18.0, npm 11.16.0

55 arquivos modificados/não commitados no momento da auditoria (assinatura, ícone, PWA) — a análise cobre o estado do disco

Sumário

[Sumário executivo](#)[Escopo e metodologia](#)[Arquitetura e superfície de ataque](#)[Achados](#)[Controles positivos](#)[Roteiro de correção](#)[Limitações e não testado](#)[Apêndice A — Matriz de cobertura](#)[Apêndices B–D](#)

Sumário executivo — auditoria de segurança do GestorFinance

Data: 14/09/2026 · **Commit:** 903f4466 (branch `main`) + 55 arquivos ainda não commitados (assinatura, ícone, PWA), auditados no estado do disco · **Modo:** análise estática completa + testes locais seguros em `localhost` com conta demo e dados canário; **produção não foi tocada.**

Veredicto (condicionado)

O sistema tem uma base sólida: o isolamento entre contas foi testado e funciona (nenhum usuário lê, altera ou apaga dados de outro), as senhas usam bcrypt, os tokens de recuperação são fortes e de uso único, a validação de entrada bloqueia injeção NoSQL e a interface não tem os padrões clássicos de XSS. **Nenhuma falha crítica foi confirmada.**

Há, porém, **2 riscos ALTOS e 6 MÉDIOS** que precisam de atenção antes de escalar a base de clientes — dois deles podem ser resolvidos hoje em menos de uma hora. Confiança geral da avaliação: **média-alta** para o código e a API (testados), **baixa** para infraestrutura da Vercel e do MongoDB Atlas (não observáveis nesta auditoria).

Os 5 riscos principais, em linguagem de negócio

- Qualquer pessoa pode entrar no sistema publicado com a conta demo** (AUD-001, ALTA). A senha está escrita no código, o repositório é público e a conta existe no banco real. Quem entra usa o produto de graça e ganha uma "porta de entrada" autenticada para outras falhas. *Correção: apagar a conta demo de produção — minutos.*
- Uma imagem maliciosa enviada como foto de cliente pode derrubar o servidor** (AUD-002, ALTA) e, em teoria, executar código nele, porque a biblioteca de imagens está numa versão com falha conhecida e o filtro confia no tipo que o navegador declara. *Correção: atualizar `sharp` / `multer` e conferir o formato pelo conteúdo — 1 hora.*
- O limite de tentativas de senha não funciona atrás da Vercel** (AUD-003, MÉDIA): todos os visitantes dividem o mesmo contador. Vinte erros de qualquer pessoa bloqueiam o login de todos por 15 minutos, e ataques de senha só são freados pela lentidão do bcrypt.
- Trocar a senha não expulsa quem já estava dentro** (AUD-004, MÉDIA): sessões valem 7 dias e não podem ser revogadas — justamente o que um cliente espera ao usar "esqueci minha senha" após suspeitar de invasão.
- A cobrança pode creditar 60 dias por um único pagamento** se duas confirmações chegarem ao mesmo tempo, e estornos não cancelam o acesso (AUD-007, MÉDIA). *Corrigir antes de ativar o Mercado

Pago.*

Caminhos de ataque compostos mais perigosos

- **AUD-001 → AUD-002:** conta demo pública elimina a única pré-condição ("estar logado") do upload de imagem malicioso: um desconhecido pode causar indisponibilidade da API de todos os clientes.
- **AUD-010 + AUD-014 + AUD-003:** descobrir quais e-mails têm conta, aceitar senhas de 6 caracteres e um limitador inoperante formam a receita clássica de **credential stuffing**; **AUD-004** garante que a sessão obtida sobreviva à troca de senha; **AUD-017** garante que ninguém perceba.
- **AUD-005:** se a borda aceitar um Host forjado, o e-mail de "esqueci minha senha" leva o cliente a um domínio do atacante — tomada de conta com um clique.

Controles fortes comprovados

Isolamento por usuário em leitura/escrita/exclusão (teste com conta canário); JWT `alg:none` rejeitado; cookie `HttpOnly + SameSite=Lax`; `bcrypt` custo 12; códigos/tokens de recuperação com HMAC, uso único, expiração e limite de tentativas; resposta uniforme no "esqueci minha senha"; validação zod em todas as rotas; erros genéricos; nenhum segredo no histórico Git; sem sinks de XSS nem source maps; helmet na API; CORS rejeita origens estranhas; autorização de admin no servidor; bloqueio de plano (402) no servidor; pagamento com preço definido no servidor e idempotente por id; imagens reprocessadas. Detalhes em `05-positive-controls.md`.

Números

Severidade	Confirmado	Provável	Hipótese	Informativo
ALTA	1 (AUD-001)	1 (AUD-002)	0	—
MÉDIA	3 (AUD-003, 004, 006)	2 (AUD-005, 007)	1 (AUD-008)	—
BAIXA	7 (AUD-009*, 010, 012, 013, 014, 015, 016)	1 (AUD-011)	0	—
INFORMATIVA	—	—	—	1 (AUD-018)

* AUD-009: gravação confirmada; efeito de remoção de arquivo alheio é provável. Controles validados: 18.

Cobertura (controles aplicáveis testados / identificados): V1 3/3 · V2 4/4 · V3 5/5 · V4 4/4 · V5 4/4 · V6 9/9 · V7 3/3 · V8 4/4 · V9 3/3 · V11 2/2 · V12 1/2 (TLS de borda não testado) · V13 4/4 · V14 2/3 (backup não testado) · V15 2/2 · V16 2/2 · V10/V17 não aplicáveis · Top 10: 9/10 (A08 não testado) · API Top 10: 10/10. Total: **61 de 64 aplicáveis (95%)**; 3 não testados por falta de acesso à infraestrutura.

Limitações críticas

Produção (site na Vercel), painel do MongoDB Atlas (rede, backups), variáveis de ambiente efetivas de produção e o fluxo real de pagamento não foram observados. Nenhum scanner (Semgrep, Gitleaks, OSV) estava instalado e nenhum foi instalado; a busca de segredos e a análise de código foram manuais. Ver `07-limitations-not-tested.md`.

Plano

- **24–48 h:** remover a conta demo de produção e endurecer `seed.ts` (AUD-001); definir `APP_URL` e `MP_WEBHOOK_SECRET` obrigatórios (AUD-005, AUD-015).

- **7 dias:** atualizar `sharp` / `multer` e validar formato por conteúdo (AUD-002); `trust proxy` + limite por conta com store compartilhado (AUD-003); aplicação atômica de pagamento com validação de valor e tratamento de estorno (AUD-007) — antes de ligar a cobrança.
- **30 dias:** revogação de sessão por versão + validade menor (AUD-004); escape e limite na busca (AUD-006); MFA e trilha de auditoria para admin (AUD-008); remover `avatarUrl` do cliente (AUD-009); tempo uniforme no login (AUD-010); cabeçalhos no `vercel.json` (AUD-011); política de senha (AUD-014); `npm audit` no build (AUD-016); logging de segurança (AUD-017); confirmar backup do Atlas (AUD-018e).
- **90 dias:** checagem de Origin/ `__Host-` (AUD-013); limpeza de CORS (AUD-012); SBOM e dependabot; plano de resposta a incidentes com rotação de `JWT_SECRET`.

01 — Escopo, autorização e metodologia

Ficha interna (preenchida pela auditoria)

Campo	Valor	Origem
PROJECT_ROOT	E:\Lucas\Projeto-Gestor (workspace único)	OBSERVADO NO PROJETO
PROJECT_NAME	GestorFinance (antes GestorPro); repositório LucasFelipeMartins/GestoFinance (público)	OBSERVADO NO PROJETO
ALLOWED_TARGETS	Arquivos do workspace; servidor de desenvolvimento local http://localhost:4000 (API) e http://localhost:5173 (SPA)	INFORMADO PELO USUÁRIO (pedido de "verificações e testes") + padrão seguro
EXCLUDED_TARGETS	Site publicado na Vercel, painel do Atlas, Mercado Pago, provedores de e-mail, o app Flutter (flutter_app/ , não implantado) e o projeto Android/Capacitor (descontinuado)	padrão do prompt
ENVIRONMENT	Código local; ambiente remoto não verificado. Atenção: o servidor local usa o mesmo banco MongoDB Atlas da produção (server/.env), por isso apenas a conta demo (dados de exemplo) e registros canário criados/removidos pela auditoria foram usados	OBSERVADO NO PROJETO
AUTHORIZATION_MODE	STATIC_ONLY estendido com testes locais seguros (sem exploração destrutiva, sem carga, sem e-mails, sem pagamentos)	INFERIDO do pedido; produção nunca acionada
AUTHORIZATION_REFERENCE	Pedido do responsável nesta sessão: "faça todas as verificações e testes de segurança do meu sistema"	INFORMADO PELO USUÁRIO
TEST_ACCOUNTS	demo@gestorpro.com (conta de exemplo do seed) e audit-canary@example.com (criada e apagada durante a auditoria); nenhuma senha reproduzida aqui	OBSERVADO
STACK_KNOWN	React 18 + Vite 7 + Tailwind + Dexie (IndexedDB) + React Query; Express 4 + Mongoose 8 + zod + jsonwebtoken + bcryptjs + helmet + express-rate-limit + multer + sharp + nodemailer + mercadopago; deploy Vercel (SPA estático + função Node em api/index.ts); MongoDB Atlas; Vercel Blob; Resend/SMTP; Mercado Pago Checkout Pro	OBSERVADO
SENSITIVE_DATA	Hipótese a partir dos schemas: nome/e-mail/hash de senha de usuários; nome, telefone, serviço, preço e foto de clientes finais; receitas, despesas, parcelas, investimentos e metas; ids de pagamento	INFERIDO
CRITICAL_FLOWS	Cadastro com código por e-mail; login; esqueci/redefinir/trocar senha; sincronização de dados (outbox); upload de avatar; assinatura (checkout, confirmação, webhook); painel admin de contas gratuitas	OBSERVADO
BUSINESS_IMPACT	Vazamento cross-usuário (fato: não encontrado); indisponibilidade; perda de receita da assinatura; tomada de contas	INFERIDO
APPLY_FILTERS	false — somente relatório	padrão

Campo	Valor	Origem
REPORT_ LANGUA GE	pt-BR	padrão

Registro de evidência (Fase 0)

- Data/hora: 2026-09-14T00:22Z (início) · Commit `903f44660ca29812abfca738e9681c89c2a68213` · branch `main` · worktree com 55 arquivos alterados (funcionalidades entregues nesta mesma sessão de trabalho: assinatura, ícone, PWA).
- Runtime: Node v24.18.0, npm 11.16.0 (Windows 11).
- Ferramentas já disponíveis: `npm audit` (consulta ao registro npm — apenas metadados de versões, sem envio de código), `git`, `curl`, `python`, Playwright-core + Chrome já instalados no scratchpad (usados só para gerar/verificar o PDF). **Não estavam instalados e não foram instalados:** Semgrep, CodeQL, Gitleaks, OSV-Scanner, Trivy, ZAP. A busca de segredos e o SAST foram manuais (grep dirigido + leitura de código).
- Diretório de resultados: `security-audit/` (sem `.env`, dumps, tokens ou dados de clientes). Evidência bruta dos testes locais: `security-audit/audit-evidence.txt` (valores de cookie redigidos).
- Limites dinâmicos: no máximo 22 requisições de login por teste; nenhuma requisição que gere e-mail (SMTP real está configurado no `.env` local) ou pagamento; nenhum padrão de ReDoS executado contra o banco compartilhado; contas/dados canário removidos ao final.

Suposições

1. O responsável é o dono do projeto (confirmado pelo pedido) — mantida.
2. O banco usado pelo servidor local é o de produção — confirmado pela URL do cluster no `.env` (valor redigido) e pela presença dos dados de exemplo; por isso os testes foram restritos a leitura, à conta demo e a canários.
3. Em produção `NODE_ENV=production` (Vercel define por padrão) — não verificado; afeta `secure` do cookie, morgan e o ramo de `resolveAppUrl`.
4. Variáveis `APP_URL`, `MP_*`, `ADMIN_EMAILS`, `CLIENT_ORIGIN` de produção — desconhecidas.

Normas e método

OWASP ASVS 5.0 (Level 2 como alvo; Level 3 sugerido para admin/pagamentos), OWASP Top 10:2025, OWASP API Security Top 10:2023, WSTG, CWE, NIST SP 800-63B-4 (senhas/recuperação), RFC 8725 (JWT). Identificadores ASVS citados apenas por capítulo/categoria: o texto exato dos requisitos não foi conferido contra a publicação — registrado como "referência exata não verificada". CVSS v4.0 calculado manualmente por achado (vetor no relatório). KEV/EPSS não consultados online (sem CVEs numerados nos avisos relevantes; GHSA indicados).

Ordem executada (Fase 5)

1. Inventário read-only (rotas, modelos, validadores, configuração, deploy, dependências).
2. Revisão manual por domínio (A–P do prompt); Q (IA) e R (mobile/WebRTC) não aplicáveis — o app Flutter existe no repositório mas não é distribuído e não foi auditado.
3. `npm audit` + `npm ls` para alcance de dependências; busca de segredos no histórico Git.
4. Testes locais seguros T01–T15 (`audit-evidence.txt`): cabeçalhos, CORS, cookie, JWT `none`, injeção de regex, erros, enumeração, BOLA com canário, mass assignment, autorização admin, gate 402, webhook

anônimo, rate limit atrás de proxy, tempo de resposta, limites de corpo.

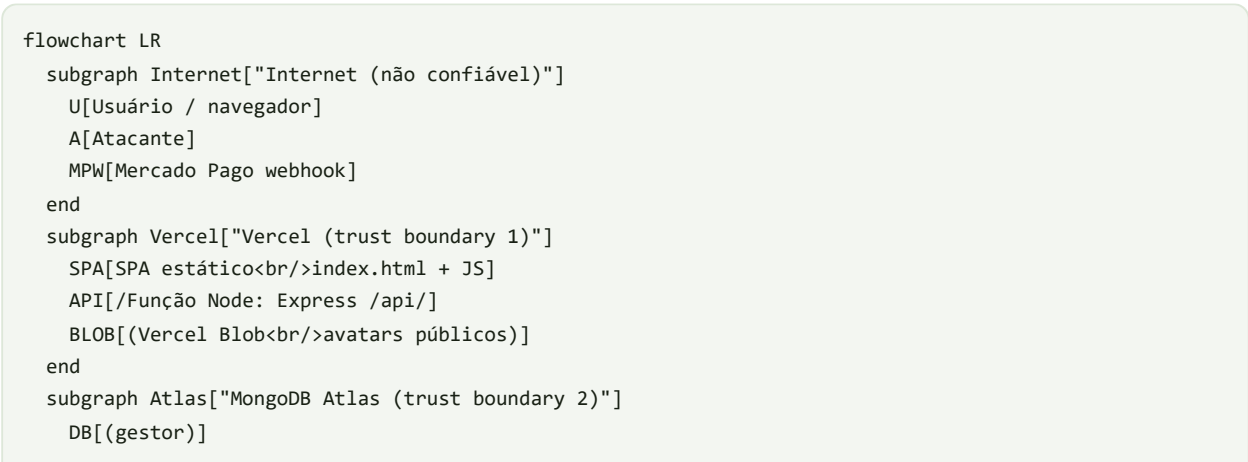
5. Triagem de falsos positivos, correlação (cadeias) e redação em duas camadas.

02 — Arquitetura real e superfície de ataque

Componentes (derivados do código)

Camada	Tecnologia	Onde
SPA (web + PWA instalável)	React 18, Vite 7, Tailwind, React Router 7, React Query, Dexie (IndexedDB), axios	<code>client/</code> — servido como estático pela Vercel (<code>vercel.json</code> <code>rewrites /((?!api/).*) → index.html</code>)
API	Express 4 em função serverless Node (<code>api/index.ts</code> → <code>server/src/app.ts</code>); helmet, cookie-parser, express.json (100 kB), morgan (dev)	rotas sob <code>/api</code>
Banco	MongoDB Atlas (Mongoose 8, <code>strictQuery</code>) — coleções <code>users</code> , <code>clients</code> , <code>tasks</code> , <code>financeentries</code> , <code>goals</code> , <code>goalcontributions</code> , <code>emailverifications</code> (TTL), <code>freeaccounts</code> , <code>payments</code>	<code>MONGO_URL</code>
Arquivos	Vercel Blob público (avatars, WebP 256×256) ou disco local <code>/uploads</code> em dev	<code>utils/avatar.ts</code>
E-mail	Resend (HTTPS) ou SMTP (nodemailer) — códigos de cadastro e links de reset	<code>services/mail.ts</code>
Pagamentos	Mercado Pago Checkout Pro (preference → redirect), confirmação por <code>POST /billing/confirm</code> e webhook	<code>services/billing.ts</code>
Identidade	Local (e-mail + senha, bcrypt 12); JWT HS256 7 dias em cookie HttpOnly (web) ou Bearer (legado nativo); sem MFA/SSO	<code>utils/jwt.ts</code> , <code>middleware/requireAuth.ts</code>
Admin	E-mails em <code>ADMIN_EMAILS</code> → <code>requireAdmin</code>	<code>controllers/admin.controller.ts</code>
Legado no repositório (não implantado)	<code>flutter_app/</code> (Flutter), <code>client/android/</code> (Capacitor)	fora do escopo

Diagrama



```

end
subgraph Ext["Terceiros"]
  MP[API Mercado Pago]
  MAIL[Resend / SMTP]
end
U -- HTTPS --> SPA
U -- cookie HttpOnly JWT --> API
A -. mesmas rotas .-> API
MPW -- POST /api/billing/webhook --> API
API -- MONGO_URL (TLS) --> DB
API -- token --> BLOB
API -- MP_ACCESS_TOKEN --> MP
API -- SMTP_*/RESEND_API_KEY --> MAIL
U -- IndexedDB local --> SPA

```

Crown jewels: `MONGO_URL` (todos os dados de todos os usuários), `JWT_SECRET` (forjar qualquer sessão), `MP_ACCESS_TOKEN` (loja no Mercado Pago), `SMTP_PASS / RESEND_API_KEY` (enviar e-mails em nome do produto), `BLOB_READ_WRITE_TOKEN`, a lista `ADMIN_EMAILS` e as contas admin.

Inventário de entradas (rotas reais — `server/src/routes/*`)

Método e rota	Auth	Plano	Observações
POST /api/auth/register/request-code	não	—	rate limit 6/15 min (por IP do proxy — AUD-003); envia e-mail; 409 revela conta existente (AUD-010)
POST /api/auth/register	não	—	exige código; cria usuário com trial
POST /api/auth/login	não	—	20/15 min; devolve cookie + token no corpo
POST /api/auth/logout	não	—	limpa cookie (sem revogação — AUD-004)
GET /api/auth/me	sim	—	inclui <code>access</code>
POST /api/auth/forgot-password	não	—	resposta uniforme; link via <code>resolveAppUrl</code> (AUD-005)
GET/POST /api/auth/reset-password	não	—	token em query (GET) e corpo (POST)
POST /api/auth/change-password	sim	—	exige senha atual
GET/POST/PUT/PATCH/DELETE /api/clients[:id]/[status]/[avatar]	sim	sim	escopo <code>userId</code> validado; avatar multipart (AUD-002, AUD-009)
GET/POST/PUT/PATCH/DELETE /api/tasks[:id]/[status]	sim	sim	escopo validado
GET/POST/PUT/DELETE /api/finance[:id]	sim	sim	escopo validado; busca regex (AUD-006)
GET/POST/PUT/DELETE /api/goals[:id], /goals/contributions/...	sim	sim	escopo validado
GET /api/billing/status, POST /api/billing/checkout, POST /api/billing/confirm	sim	—	confirm consulta MP (AUD-007)
POST /api/billing/webhook	não	—	assinatura opcional (AUD-015)
GET/POST/DELETE /api/admin/free-accounts[:email]	admin	—	sem MFA/log (AUD-008)
GET /api/health	não	—	<code>{status: 'ok'}</code>

Método e rota	Auth	Plano	Observações
GET /uploads/*	não	—	só em dev/disco

Comparação com a documentação (README) e com a interface: todas as rotas usadas pelo SPA existem no servidor; não há rotas de debug, versões antigas ou endpoints sem uso (o antigo caminho `/lucros` é redirecionado no cliente). Drift: nenhum identificado.

Egress (saídas do servidor)

Atlas (banco), api.resend.com ou SMTP configurado, api.mercadopago.com, Vercel Blob. Nenhum destino é derivado de entrada do usuário (SSRF não aplicável — controle validado).

Fronteiras de confiança e observações

- **Navegador ↔ API:** tudo que vem do cliente é validado com zod; a decisão de plano e o preço são do servidor. Exceção: `avatarUrl` (AUD-009) e `updatedAt` (last-write-wins, afeta só o próprio usuário).
- **Proxy da Vercel ↔ Express:** o IP real não é reconhecido (AUD-003); o Host/Origin da requisição alimenta links de e-mail quando `APP_URL` falta (AUD-005).
- **Terceiros ↔ API:** o webhook não é confiado sem consulta ao Mercado Pago (bom), mas aceita chamadas anônimas quando sem segredo (AUD-015).
- **Dispositivo do usuário:** IndexedDB guarda todos os dados até "Sair" (AUD-018d); `access` também é cacheado, mas o servidor reforça o 402.

Matriz ator × objeto × ação (fluxos de maior risco)

Ator	Objeto	ler	criar	editar	excluir	Resultado
Anônimo	cliente/tarefa/finança de A	401	401	401	401	validado (T08)
Usuário B (canário)	cliente de A	404	—	404	404	validado (T08)
Usuário A	próprio <code>avatarUrl</code> arbitrário	—	—	200 aceito	—	AUD-009
Usuário com trial vencido	qualquer dado	402	402	402	402	validado (T10)
Usuário comum	<code>/api/admin/*</code>	403	403	—	403	validado (T09)
Admin (sem MFA)	contas gratuitas	200	201	—	204	AUD-008
Anônimo	webhook de pagamento	—	200 (consulta MP)	—	—	AUD-015

04 — Achados

Ordenados por severidade. Cada achado tem duas camadas: explicação didática e detalhes técnicos. Evidências brutas em `audit-evidence.txt` (T01–T15).

AUD-001 — Conta de demonstração com senha pública no repositório e ativa no banco de produção

- **Status:** CONFIRMADO · **Confiança:** alta · **Severidade:** ALTA
- **CVSS v4.0:** 7.1 — `CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N`
- **CWE:** CWE-798 Uso de credenciais embutidas; CWE-1392 Credenciais padrão · **OWASP Top 10:2025:** A07:2025 Authentication Failures · **API Top 10:2023:** API2:2023 Broken Authentication · **ASVS:** V6 Authentication (credenciais padrão/conhecidas) — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável (sem CVE)
- **Onde está:** `server/src/seed.ts:12-13` (DEMO_EMAIL / DEMO_PASSWORD); banco `gestor` no Atlas; repositório público em `github.com/LucasFelipeMartins/GestoFinance`
- **Owner / prazo / esforço:** Dono do produto (Lucas) · URGENTE, 24–48h · baixo

Camada 1 — Entenda a brecha

O que encontramos. O script de carga de exemplo grava no código a conta `demo@gestorpro.com` com senha fixa; o repositório é público (HTTP 200 sem login) e essa conta existe e aceita login no banco de produção usado pelo site.

Como deveria funcionar. Contas de demonstração devem existir apenas em bancos de teste, com senha gerada na hora e nunca escrita no código. Se um dado de exemplo precisar ir para produção, ele deve ser removido antes de abrir o sistema ao público.

O que está acontecendo. Durante os testes locais o servidor de desenvolvimento (ligado ao mesmo banco Atlas da produção) devolveu HTTP 200 para `POST /api/auth/login` com as credenciais que estão em `seed.ts`. O repositório GitHub responde 200 para visitantes anônimos, então a senha é pública.

Por que isso é uma brecha. Autenticação existe para provar quem está pedindo. Uma senha que qualquer pessoa pode ler no GitHub não prova nada: qualquer visitante vira usuário autenticado do sistema real sem criar conta, sem confirmar e-mail e sem passar pela assinatura. Toda proteção que exige 'estar logado' (upload de imagem, buscas, criação de registros) deixa de ser barreira para quem usa essa conta.

Sequência causal. Senha em código público → conta correspondente presente no banco de produção → `POST /api/auth/login` aceita → sessão válida de 7 dias emitida → acesso a todos os endpoints autenticados e ao plano sem pagar.

Como alguém poderia abusar. Alguém lê `seed.ts` no GitHub, entra no site publicado com `demo@gestorpro.com` e a senha do arquivo, e passa a usar o produto gratuitamente ou a explorar falhas que exigem login (AUD-002, AUD-006) sem deixar rastro ligado a uma identidade real. Vários atacantes compartilhando a mesma conta também podem apagar dados uns dos outros nela.

O que foi demonstrado. Login bem-sucedido (200) em localhost contra o banco de produção com as credenciais do arquivo; repositório público confirmado.

O que pode acontecer. Uso gratuito do SaaS; base autenticada para outros ataques; alteração/remoção dos dados de exemplo; reputação. Não há acesso a dados de outras contas por esse caminho (escopo por usuário validado em AUD-CV-01).

Por que recebeu este nível. ALTA e não CRÍTICA: a conta é comum (não administrador) e o isolamento por usuário impede alcançar dados de terceiros. É mais que MÉDIA porque remove completamente a barreira de autenticação em produção para qualquer pessoa, sem interação da vítima.

O que faria o nível subir ou descer. Sobe para CRÍTICA se o e-mail demo for colocado em ADMIN_EMAILS ou se surgir uma falha explorável apenas autenticada com impacto amplo. Desce para BAIXA se a conta for removida de produção ou o repositório ficar privado com a senha trocada.

Como corrigir. 1) Apagar o usuário demo do banco de produção (ou trocar a senha para uma gerada e não publicada). 2) Em seed.ts, recusar execução quando NODE_ENV=production e gerar senha aleatória impressa apenas no console. 3) Avaliar tornar o repositório privado ou, se ficar público, garantir que nada nele funcione em produção. 4) Considerar exigir troca de senha no primeiro login para contas provisionadas.

Como confirmar a correção. 1) POST /api/auth/login com demo@gestorpro.com/senha do arquivo em produção → 401. 2) NODE_ENV=production npm run seed → aborta com mensagem. 3) Busca no repositório pela senha antiga do seed → nenhum resultado.

Camada 2 — Detalhes técnicos

Evidência (redigida): T03/T08 em audit-evidence: POST /api/auth/login → 200 e Set-Cookie (valor redigido).
 curl -s -o /dev/null -w '%{http_code}' https://github.com/LucasFelipeMartins/GestoFinance → 200.

Risco residual: Contas de teste criadas manualmente continuam sendo risco se tiverem senhas fracas; revisar periodicamente usuários sem atividade.

AUD-002 — Upload de avatar alcança decodificadores de imagem vulneráveis (sharp/libheif) e multer com falhas de DoS

- **Status:** PROVÁVEL · **Confiança:** média · **Severidade:** ALTA
- **CVSS v4.0:** 7.5 — CVSS:4.0/AV:N/AC:L/AT:P/PR:L/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N
- **CWE:** CWE-1395 Dependência com vulnerabilidade conhecida; CWE-122 Heap overflow (libheif); CWE-400 Consumo descontrolado de recursos (multer) · **OWASP Top 10:2025:** A03:2025 Software Supply Chain Failures · **API Top 10:2023:** API8:2023 Security Misconfiguration · **ASVS:** V5 File Handling / V15 Secure Coding — referência exata não verificada
- **CISA KEV / EPSS:** não consultado (advisories GitHub: GHSA-rgj7-g3m4-5g8c, GHSA-wc9g-mqfw-jrwm, GHSA-qfvm-cv95-jqjf, GHSA-535w-7cp7-47q4)
- **Onde está:** server/src/middleware/uploadAvatar.ts:6-18 (fileFilter por MIME declarado); server/src/utis/avatar.ts:9-11 (sharp(buffer) decodifica pelo conteúdo); package-lock: sharp 0.35.3, multer 2.2.0
- **Owner / prazo / esforço:** Desenvolvimento · CURTO, até 7 dias · baixo

Camada 1 — Entenda a brecha

O que encontramos. Qualquer usuário logado pode enviar um arquivo em POST /api/clients/:id/avatar; o filtro só confere o tipo que o navegador declara, e a biblioteca sharp decide o formato pelo conteúdo real — inclusive HEIF/AVIF, cujo decodificador (libheif) tem falhas de corrupção de memória corrigidas em sharp 0.35.4. O multer instalado (2.2.0) tem três avisos de negação de serviço por campos multipart maliciosos.

Como deveria funcionar. Arquivos vindos do usuário devem ser decodificados apenas nos formatos que o produto aceita (JPG, PNG, WebP), por uma biblioteca atualizada, com limites de pixels, dentro de um processo isolado; o formato deve ser conferido pelos primeiros bytes, não pelo cabeçalho enviado pelo cliente.

O que está acontecendo. ALLOWED_MIME_TYPES compara file.mimetype (controlado pelo cliente). sharp(buffer).resize(...) não restringe formato de entrada; o binário pré-compilado presente suporta entrada HEIF (sharp.format.heif.input.buffer === true, verificado). npm audit lista sharp <0.35.4 (libheif) e multer ≤2.2.0 (DoS).

Por que isso é uma brecha. Um arquivo que se declara `image/png` mas contém um HEIF malicioso passa pelo filtro e chega ao decodificador vulnerável. Em uma função serverless, uma falha de memória derruba a execução (negação de serviço) e, em cenários de heap overflow, pode permitir execução de código no ambiente que guarda MONGO_URL, JWT_SECRET e o token do Mercado Pago. O multer, por sua vez, pode ser travado com nomes de campo especialmente montados antes mesmo de a imagem ser processada.

Sequência causal. Usuário autenticado (ou AUD-001) → multipart com MIME falso e conteúdo HEIF/AVIF ou campos malformados → fileFilter aceita → sharp/libheif decodifica → corrupção de memória ou esgotamento de recursos → indisponibilidade ou, potencialmente, execução de código com acesso aos segredos do servidor.

Como alguém poderia abusar. Uma conta comum (ou a demo pública) envia repetidamente um AVIF/HEIF malformado como avatar de um cliente próprio; a função da Vercel falha a cada tentativa e o serviço fica instável para todos; em versão explorável da libheif o invasor tenta obter execução remota.

O que foi demonstrado. Alcance confirmado: filtro por MIME declarado e suporte HEIF no binário instalado. A exploração (arquivo malicioso) NÃO foi executada — não é um teste seguro sem ambiente isolado.

O que pode acontecer. Indisponibilidade da API para todos os usuários; comprometimento do ambiente serverless e dos segredos (banco, JWT, pagamentos) em caso de exploração bem-sucedida da corrupção de memória.

Por que recebeu este nível. ALTA: pré-condição real (conta autenticada, hoje trivial por AUD-001), correção simples, impacto potencial de DoS confirmado pelos avisos e possibilidade de RCE não demonstrada. Não é CRÍTICA porque a execução de código não foi comprovada nem há PoC pública verificada para este uso.

O que faria o nível subir ou descer. Sobe para CRÍTICA se existir exploit público confiável para a versão instalada ou se o upload ficar disponível sem login. Desce para MÉDIA após atualizar sharp/multer e restringir formatos de entrada.

Como corrigir. 1) `npm install sharp@^0.35.4 multer@latest -w server` e reexecutar `npm audit`. 2) Antes de processar, conferir o formato real: `const meta = await sharp(buffer, { limitInputPixels: 24_000_000 }).meta(); if (!['jpeg', 'png', 'webp'].includes(meta.format)) rejeitar`. 3) Manter `limits.fileSize`, adicionar `limits.fields/parts` no multer. 4) Executar `npm audit` no pipeline de deploy (Vercel Build) e falhar em vulnerabilidades altas alcançáveis.

Como confirmar a correção. 1) Upload de PNG válido → 200 e avatar WebP salvo. 2) Upload de AVIF/HEIF com MIME `image/png` → 400 'Formato não suportado'. 3) Upload de SVG/HTML → 400. 4) `npm audit --audit-level=high` sem itens em sharp/multer.

Camada 2 — Detalhes técnicos

Evidência (redigida): `npm audit --json` (sharp <0.35.4, multer ≤2.2.0, cvss 7.5 nos avisos multer); `node -e "require('sharp').format.heif.input" → {file:true,buffer:true,stream:true}`; uploadAvatar.ts linhas 6-18.

Risco residual: Decodificadores JPEG/PNG/WebP também recebem correções periódicas; manter rotina de atualização mensal.

AUD-003 — Limite de tentativas de login inoperante atrás do proxy: todos os visitantes compartilham um único contador e cada instância serverless tem o seu

- **Status:** CONFIRMADO · **Confiança:** alta · **Severidade:** MÉDIA
- **CVSS v4.0:** 6.3 — CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:L/SC:N/SI:N/SA:N
- **CWE:** CWE-307 Restrição inadequada de tentativas de autenticação; CWE-348 Uso de fonte menos confiável para o IP · **OWASP Top 10:2025:** A07:2025 Authentication Failures · **API Top 10:2023:** API4:2023 Unrestricted Resource Consumption · **ASVS:** V6 Authentication (anti-automação) — referência exata não verificada

- **CISA KEV / EPSS:** não aplicável
- **Onde está:** server/src/app.ts (sem `app.set('trust proxy', ...)`); server/src/routes/auth.routes.ts:18-35 (express-rate-limit com store em memória, 20/15min por `req.ip`)
- **Owner / prazo / esforço:** Desenvolvimento · CURTO, até 7 dias · médio

Camada 1 — Entenda a brecha

O que encontramos. O limitador de tentativas usa o endereço IP que o Express enxerga. Na Vercel, as requisições chegam por um proxy; sem `trust proxy`, o Express vê sempre o IP do proxy, então todos os usuários caem no mesmo balde de 20 tentativas por 15 minutos. Além disso o contador vive na memória de cada instância serverless, que nasce e morre a todo momento.

Como deveria funcionar. Cada cliente deve ter seu próprio limite (por IP real e por conta), guardado em um lugar compartilhado entre instâncias (banco ou Redis), com bloqueio progressivo por conta após várias falhas.

O que está acontecendo. Teste T12: 22 logins errados enviados com cabeçalhos X-Forwarded-For diferentes (simulando 22 visitantes) — o servidor devolveu 401 até a 18ª e 429 a partir daí, provando que ignora o cabeçalho e agrupa todos em um só contador. O código não define `trust proxy`.

Por que isso é uma brecha. Dois efeitos opostos e ambos ruins: (1) um atacante que acerta uma instância nova ou espera o reinício tem 20 tentativas por instância, e como a Vercel abre várias instâncias em paralelo o limite real é multiplicado; (2) 20 erros de senha de quaisquer pessoas no mundo bloqueiam o login de todos os clientes daquela instância por 15 minutos — um usuário mal-intencionado pode negar o login a todo mundo com 20 requisições. O mesmo vale para o envio de códigos de cadastro (6/15min), que passa a ser um limite global: bastam 6 cadastros legítimos para o 7º ser recusado.

Sequência causal. Requisição via proxy → `req.ip` = IP do proxy (trust proxy desligado) → chave do rate limit igual para todos → contador por instância em memória → (a) força bruta distribuída entre instâncias; (b) 429 para usuários legítimos após 20 falhas alheias.

Como alguém poderia abusar. Alguém envia 20 senhas erradas para qualquer e-mail: clientes reais recebem 'Muitas tentativas' por 15 minutos naquela instância. Ou tenta senhas comuns contra um e-mail conhecido, repetindo a cada nova instância/janela sem bloqueio por conta.

O que foi demonstrado. Compartilhamento do contador entre 'IPs' diferentes (T12) e ausência de bloqueio por conta.

O que pode acontecer. Comprometimento de contas com senhas fracas (mínimo de 6 caracteres, AUD-014); indisponibilidade do login/cadastro para clientes legítimos; abuso do envio de e-mails.

Por que recebeu este nível. MÉDIA: impacto real (takeover de senhas fracas, DoS de login) mas exige volume e é limitado pelo bcrypt (~280 ms por tentativa) e pela ausência de enumeração fácil de senhas. Não é ALTA porque não há bypass de autenticação direto.

O que faria o nível subir ou descer. Sobe para ALTA se combinada com senhas fracas conhecidas ou lista de e-mails vazada. Desce para BAIXA com limite por conta + store compartilhado + bloqueio progressivo.

Como corrigir. 1) `app.set('trust proxy', 1)` (Vercel coloca o IP real em X-Forwarded-For) e validar `req.ip` no log. 2) Store compartilhado: `rate-limit-mongo` ou coleção própria com TTL, já que o projeto tem Mongo. 3) Limite adicional por conta (e-mail) em login e request-code, com backoff exponencial e alerta ao titular após N falhas. 4) Devolver `Retry-After`. 5) Considerar desafio (CAPTCHA) apenas após falhas repetidas por conta.

Como confirmar a correção. 1) 21 tentativas com o mesmo XFF → 429 na 21ª; 2) tentativas com XFF diferentes → não se somam; 3) reiniciar o servidor não zera o contador (store compartilhado); 4) 10 falhas na mesma conta → 429 mesmo com IPs diferentes.

Camada 2 — Detalhes técnicos

Evidência (redigida): audit-evidence T12: sequência '401×18, 429×4' com X-Forwarded-For distintos. Código: ausência de `trust proxy ; rateLimit({ windowMs: 15*60*1000, limit: 20 })` sem `store`.

Risco residual: Ataques distribuídos de baixa velocidade continuam possíveis; MFA (AUD-008) e senhas fortes (AUD-014) reduzem o impacto.

AUD-004 — Sessões (JWT de 7 dias) não podem ser revogadas: trocar/redefinir a senha ou sair não invalida tokens já emitidos

- **Status:** CONFIRMADO · **Confiança:** alta · **Severidade:** MÉDIA
- **CVSS v4.0:** 5.9 — `CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N`
- **CWE:** CWE-613 Expiração de sessão insuficiente; CWE-287 · **OWASP Top 10:2025:** A07:2025 Authentication Failures · **API Top 10:2023:** API2:2023 Broken Authentication · **ASVS:** V7 Session Management (invalidação/expiração) — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** `server/src/utils/jwt.ts` (HS256, expiresIn 7d, sem `jti` /versão); `server/src/middleware/requireAuth.ts:26-38` (verifica só assinatura/expiração); `server/src/controllers/auth.controller.ts` (resetPassword/changePassword só rehash da senha; logout só limpa cookie)
- **Owner / prazo / esforço:** Desenvolvimento · MÉDIO, até 30 dias · médio

Camada 1 — Entenda a brecha

O que encontramos. O login gera um token assinado que vale 7 dias e o servidor só confere a assinatura e a validade. Não existe lista de sessões, versão de senha no token nem expiração por inatividade. Logout apaga o cookie do navegador, mas o token continua aceito se alguém o tiver copiado.

Como deveria funcionar. Quando a senha muda (troca voluntária ou 'esqueci minha senha'), todas as sessões anteriores devem parar de valer; sair deve invalidar a sessão no servidor; sessões longas devem exigir atividade recente.

O que está acontecendo. `verifyToken` apenas `jwt.verify(token, secret)`; nenhum campo do usuário (ex.: `passwordChangedAt`, `sessionVersion`) é comparado. `resetPassword` faz `res.clearCookie` (só no navegador que pediu) e não altera nada que invalide outros tokens.

Por que isso é uma brecha. O cenário-alvo de 'esqueci minha senha' é justamente quando a pessoa suspeita que outra pessoa está usando sua conta. Aqui, após redefinir a senha, o invasor com o token antigo (cookie roubado, dispositivo compartilhado, app no celular perdido) continua dentro por até 7 dias. Como o token está em cookie `HttpOnly` no web, o roubo exige acesso ao dispositivo ou ao tráfego, o que limita — mas não elimina — o risco.

Sequência causal. Token emitido (7d) → cópia obtida por terceiro (dispositivo compartilhado/perdido, malware) → titular redefine senha → servidor não relaciona senha a tokens → token antigo segue válido até expirar.

Como alguém poderia abusar. A pessoa usa o app em um computador de terceiros e esquece de sair; ao perceber, troca a senha do celular. O terceiro continua com acesso completo por até uma semana.

O que foi demonstrado. Por leitura de código (não foi necessário teste dinâmico: não existe caminho de invalidação).

O que pode acontecer. Permanência de invasor após recuperação da conta; impossibilidade de encerrar sessões em incidente (ex.: vazamento do `JWT_SECRET` exigiria rotacionar a chave e deslogar todo mundo).

Por que recebeu este nível. MÉDIA: exige que o token já tenha sido obtido; impacto limitado à conta afetada, mas anula a principal defesa do usuário (trocar senha).

O que faria o nível subir ou descer. Sobe para ALTA se o token passar a ser guardado em localStorage ou exposto em logs/URLs. Desce para BAIXA com versão de sessão + expiração curta com renovação.

Como corrigir. 1) Adicionar `sessionVersion` (ou `passwordChangedAt`) ao usuário, incluir no payload do JWT e rejeitar em `requireAuth` quando divergir; incrementar em reset/troca de senha e em 'sair de todos os dispositivos'. 2) Reduzir a validade para ~1h com renovação silenciosa via refresh token rotativo em cookie, ou manter 7d absoluto + 24h de inatividade. 3) Registrar `iss / aud` e validar em `jwt.verify` com `algorithms: ['HS256']`. 4) Ao rotacionar JWT_SECRET, aceitar a chave antiga por um curto período ou forçar novo login com aviso.

Como confirmar a correção. 1) Login → token A; redefinir senha → GET /api/auth/me com token A → 401. 2) Logout → token → 401. 3) Token com `alg: HS512` assinado com a mesma chave → 401 (algoritmo restrito).

Camada 2 — Detalhes técnicos

Evidência (redigida): `jwt.ts` e `requireAuth.ts` citados; `auth.controller.ts` `resetPassword` linhas `'user.passwordHash = ...; res.clearCookie'`.

Risco residual: Tokens roubados valem até a próxima invalidação; sem MFA o reset por e-mail continua sendo o fator único.

AUD-005 — Link de redefinição de senha montado a partir dos cabeçalhos Origin/Host da requisição (envenenamento de link)

- **Status:** PROVÁVEL · **Confiança:** média · **Severidade:** MÉDIA
- **CVSS v4.0:** 5.3 — `CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:P/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N`
- **CWE:** CWE-640 Mecanismo fraco de recuperação de senha; CWE-20 · **OWASP Top 10:2025:** A07:2025 Authentication Failures · **API Top 10:2023:** API8:2023 Security Misconfiguration · **ASVS:** V6 Authentication (recuperação) / V13 Configuration — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** `server/src/utils/appUrl.ts:10-27 (resolveAppUrl)`; `server/src/controllers/auth.controller.ts forgotPassword ( ${resolveAppUrl(req)}/redefinir-senha?token=... )`; APP_URL opcional em `.env.example`
- **Owner / prazo / esforço:** Desenvolvimento / Deploy / CURTO, até 7 dias · baixo

Camada 1 — Entenda a brecha

O que encontramos. Quando APP_URL não está definido, o endereço usado no e-mail de 'esqueci minha senha' é calculado a partir do que a requisição informa: Origin (se igual a `https://<Host>`) ou, em último caso, o próprio Host. Esses cabeçalhos são controlados por quem faz a requisição.

Como deveria funcionar. Links enviados por e-mail devem usar um endereço fixo e confiável configurado no servidor, nunca derivado da requisição.

O que está acontecendo. `if (origin && !isLocalOrigin && (sameOrigin || env.clientOrigins.includes(origin))) return origin;` com `sameOrigin = origin === https://${host}` — ambos vindos do cliente. Em desenvolvimento qualquer Origin é aceito. Mitigação parcial: a borda da Vercel só encaminha requisições cujo Host pertença ao projeto, o que reduz o ataque em produção; não foi possível verificar outros cabeçalhos (X-Forwarded-Host) nem a lista CLIENT_ORIGIN efetiva.

Por que isso é uma brecha. 'Envenenamento de link de reset' é quando o atacante pede a redefinição em nome da vítima, mas com um cabeçalho Host/Origin apontando para um domínio dele; o e-mail legítimo chega à vítima com um link para o site do atacante contendo o token; ao clicar, o token é entregue ao atacante, que redefine a senha. A defesa correta é ignorar cabeçalhos e usar APP_URL.

Sequência causal. POST /api/auth/forgot-password com Host/Origin do atacante → resolveAppUrl devolve domínio do atacante → e-mail à vítima com link envenenado → vítima clica → token de reset no servidor do atacante → POST /reset-password pelo atacante → conta tomada.

Como alguém poderia abusar. Atacante conhece o e-mail da vítima; envia o pedido de reset com cabeçalhos manipulados; a vítima recebe um e-mail idêntico ao real (mesmo remetente e visual) e clica.

O que foi demonstrado. Somente leitura de código; não foi enviado e-mail em teste (custo/ruído e servidor local com SMTP real configurado).

O que pode acontecer. Tomada de conta de qualquer usuário cujo e-mail seja conhecido, condicionada a a borda aceitar o Host forjado e ao clique da vítima.

Por que recebeu este nível. MÉDIA: exige interação da vítima e depende de a infraestrutura encaminhar o Host manipulado (na Vercel isso provavelmente não ocorre, o que sustenta PROVÁVEL e não CONFIRMADO). O impacto seria alto (tomada de conta).

O que faria o nível subir ou descer. Sobe para ALTA se um domínio próprio com wildcard/CNAME aceitar hosts arbitrários ou se X-Forwarded-Host passar a ser usado. Desce para INFORMATIVA ao fixar APP_URL.

Como corrigir. 1) Definir APP_URL em produção e tornar obrigatório quando NODE_ENV=production (falhar no boot se ausente). 2) Remover o ramo que aceita `origin` genérico; manter apenas allowlist estrita (CLIENT_ORIGIN) e nunca o Host. 3) Manter o token de reset fora de logs (já em query, considerar POST ou `history.replaceState` na página).

Como confirmar a correção. 1) Pedido de reset com `Host: evil.example / Origin: https://evil.example` → link no e-mail (ou devLink) começa com APP_URL. 2) Boot em produção sem APP_URL → erro claro.

Camada 2 — Detalhes técnicos

Evidência (redigida): appUrl.ts (trecho citado); README lista APP_URL como opcional.

Risco residual: Nenhum relevante após fixar a URL.

AUD-006 — Texto de busca vira expressão regular sem escape nem limite (erro 500 e risco de lentidão no banco)

- **Status:** CONFIRMADO · **Confiança:** alta · **Severidade:** MÉDIA
- **CVSS v4.0:** 5.3 — `CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N`
- **CWE:** CWE-1333 Complexidade ineficiente de regex (ReDoS); CWE-20 Validação de entrada imprópria · **OWASP Top 10:2025:** A05:2025 Injection · **API Top 10:2023:** API4:2023 Unrestricted Resource Consumption · **ASVS:** V2 Validation / V4 API — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** server/src/controllers/client.controller.ts:23; task.controller.ts:67; finance.controller.ts:70 (`new RegExp(query.search.trim(), 'i')`); validators `search: z.string().trim().optional()` sem tamanho máximo
- **Owner / prazo / esforço:** Desenvolvimento · MÉDIO, até 30 dias · baixo

Camada 1 — Entenda a brecha

O que encontramos. O que o usuário digita na busca é usado diretamente como expressão regular na consulta ao MongoDB, sem escapar caracteres especiais nem limitar o tamanho.

Como deveria funcionar. Buscas de texto livre devem escapar os caracteres de regex (ou usar índice de texto), limitar o comprimento (ex.: 100 caracteres) e responder 400 para entradas inválidas.

O que está acontecendo. T05: `GET /api/clients?search=(` → 500 'Erro interno' (regex inválida lança exceção). T15: busca com 5.000 caracteres aceita (200). `search=.*` funciona como curinga. Os resultados continuam

restritos ao próprio usuário (filtro `userId` é AND com o `$or`), então não há vazamento de dados.

Por que isso é uma brecha. Uma regex com repetições aninhadas pode obrigar o banco a testar bilhões de combinações por documento ('ReDoS'); como a consulta roda no cluster Atlas compartilhado por todos os usuários, um único cliente autenticado pode degradar o serviço para os demais. O erro 500 mostra falta de validação e polui os logs.

Sequência causal. Parâmetro `search` do cliente → `new RegExp(...)` sem escape/limite → `$or` de regex no MongoDB → padrão patológico avaliado em todos os documentos do usuário → CPU do banco consumida / exceção 500.

Como alguém poderia abusar. Usuário logado (ou a conta demo pública) faz várias requisições com um padrão como `(a+)+$` seguido de texto longo contra suas próprias listas; o banco fica lento para todos.

O que foi demonstrado. 500 com `( ;` aceitação de 5.000 caracteres. O padrão de ReDoS NÃO foi executado contra o Atlas por ser um teste potencialmente danoso ao ambiente compartilhado (HIPÓTESE quanto ao tempo de execução real).

O que pode acontecer. Degradação/indisponibilidade do banco compartilhado; custos no Atlas.

Por que recebeu este nível. MÉDIA: precisa de conta autenticada e o dano é de disponibilidade; sem vazamento. Não BAIXA porque o vetor é trivial e atinge um recurso compartilhado.

O que faria o nível subir ou descer. Sobe para ALTA se um teste em ambiente isolado demonstrar segundos de CPU por requisição. Desce para BAIXA com escape + limite.

Como corrigir. 1) Escapar: `const safe = search.replace(/[.*+?^${}()|[\]\\]/g, '\\$&')` antes de `new RegExp` p. 2) `search: z.string().trim().max(100)`. 3) Preferir `$regex` com prefixo/índice ou Atlas Search para bases maiores. 4) Mapear `SyntaxError` para 400.

Como confirmar a correção. 1) `?search=(` → 200 lista vazia (ou 400), nunca 500. 2) `?search=` com 101 caracteres → 400. 3) Busca por 'a.b' encontra apenas texto literal 'a.b'.

Camada 2 — Detalhes técnicos

Evidência (redigida): audit-evidence T05 e T15.

Risco residual: Nenhum relevante.

AUD-007 — Aplicação de pagamento não é atômica (pode creditar 30 dias duas vezes) e não valida valor/moeda nem estornos

- **Status:** PROVÁVEL · **Confiança:** média · **Severidade:** MÉDIA
- **CVSS v4.0:** 4.8 — `CVSS:4.0/AV:N/AC:H/AT:P/PR:L/UI:N/VC:N/VI:L/VA:N/SC:N/SI:L/SA:N`
- **CWE:** CWE-362 Condição de corrida (TOCTOU); CWE-841 Aplicação imprópria de fluxo de negócio · **OWASP Top 10:2025:** A06:2025 Insecure Design · **API Top 10:2023:** API6:2023 Unrestricted Access to Sensitive Business Flows · **ASVS:** V2 Business Logic — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** `server/src/services/billing.ts applyPaymentById (findOneAndUpdate → if (status !== 'approved' || record.appliedAt) return user.save() → record.save())`
- **Owner / prazo / esforço:** Desenvolvimento · CURTO, até 7 dias (antes de ativar a cobrança) · baixo

Camada 1 — Entenda a brecha

O que encontramos. Quando um pagamento é confirmado, o servidor verifica se ele já foi aplicado e só depois grava; entre a verificação e a gravação, uma segunda chamada (webhook + retorno da página de pagamento ao mesmo tempo, ou duas abas) pode passar pela mesma verificação e somar 30 dias mais uma vez. O código

também não confere se o valor pago corresponde ao preço nem trata pagamentos estornados ou contestados depois de aprovados.

Como deveria funcionar. Créditos de assinatura devem ser aplicados com uma operação atômica no banco (marcar como aplicado e só então creditar, uma única vez), validar `transaction_amount / currency_id` contra o preço configurado e reverter o acesso em `refunded / charged_back`.

O que está acontecendo. Há dois caminhos que chamam `applyPaymentById` (webhook e POST `/billing/confirm`) sem lock; a idempotência depende de `record.appliedAt` lido antes da escrita. Não há verificação de `transaction_amount` nem tratamento de status pós-aprovação.

Por que isso é uma brecha. 'Race condition' é quando duas execuções simultâneas leem o mesmo estado antes que qualquer uma o altere. Aqui o usuário controla o momento de chamar `/billing/confirm` e o Mercado Pago o momento do webhook; disparando ambos ao mesmo tempo (ou dois confirms em paralelo), um pagamento de R\$ 11,90 pode virar 60 dias. Estornos não reduzirem o acesso significa que um cartão contestado mantém o serviço ativo.

Sequência causal. Pagamento aprovado → duas requisições concorrentes chamam `applyPaymentById` → ambas leem `appliedAt` vazio → ambas somam 30 dias em `paidUntil` → 60 dias por um pagamento.

Como alguém poderia abusar. Após pagar, o usuário dispara 5 requisições simultâneas a POST `/api/billing/confirm` com o mesmo `payment_id`; conforme o timing, obtém meses extras.

O que foi demonstrado. Somente por análise de código: exigiria pagamento real/credenciais do Mercado Pago para um teste dinâmico.

O que pode acontecer. Perda de receita (dias gratuitos indevidos); acesso mantido após estorno.

Por que recebeu este nível. MÉDIA: impacto financeiro limitado por usuário e exige timing; não afeta dados de terceiros.

O que faria o nível subir ou descer. Sobe se o preço passar a variar por plano (valor não validado) ou se surgirem preferências criadas fora do servidor. Desce para BAIXA com update atômico.

Como corrigir. 1) Aplicar de forma atômica: `Payment.findOneAndUpdate({ providerPaymentId, appliedAt: null, status: 'approved' }, { $set: { appliedAt: now } })` e só creditar se o documento retornado existir. 2) Validar `payment.transaction_amount >= priceMonthly` e `currency_id === 'BRL'`; registrar divergências. 3) Tratar `refunded`, `charged_back` e `cancelled` após aplicado: reduzir `paidUntil` pelo período creditado. 4) Rate limit em `/billing/confirm` e `/billing/webhook`.

Como confirmar a correção. 1) Dois `confirm` concorrentes com o mesmo id (mock do SDK) → `paidUntil` avança 30 dias uma única vez. 2) Pagamento aprovado com valor menor que o preço → não aplicado e logado. 3) Evento `refunded` → `paidUntil` recua.

Camada 2 — Detalhes técnicos

Evidência (redigida): `billing.ts`, função `applyPaymentById` (linhas do bloco `if (status !== 'approved' || record.appliedAt) return ...` até `record.save()`).

Risco residual: Fraude de cartão continua possível como em qualquer checkout; monitorar chargebacks no painel do Mercado Pago.

AUD-008 — Administrador protegido apenas por senha (sem MFA) e sem trilha de auditoria

- **Status:** HIPÓTESE · **Confiança:** alta (o desenho está claro no código; a exploração depende de comprometer a senha) · **Severidade:** MÉDIA
- **CVSS v4.0:** 5.1 — `CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:L/VI:L/VA:N/SC:L/SI:L/SA:N`

- **CWE:** CWE-308 Uso de autenticação de fator único; CWE-778 Registro insuficiente · **OWASP Top 10:2025:** A07:2025 Authentication Failures; A09:2025 Security Logging and Alerting Failures · **API Top 10:2023:** API5:2023 Broken Function Level Authorization (contexto) · **ASVS:** V6 (MFA para funções administrativas) / V16 Logging — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** server/src/controllers/admin.controller.ts (requireAdmin por e-mail em ADMIN_EMAILS); server/src/config/env.ts adminEmails; ausência de log em addFreeAccount/removeFreeAccount
- **Owner / prazo / esforço:** Desenvolvimento · MÉDIO, até 30 dias · médio

Camada 1 — Entenda a brecha

O que encontramos. Quem estiver na lista ADMIN_EMAILS pode liberar acesso gratuito a qualquer e-mail. Esse poder depende só da senha da conta (mínimo 6 caracteres) e de uma sessão de 7 dias; não há segundo fator nem registro de quem liberou o quê.

Como deveria funcionar. Funções administrativas de um SaaS devem exigir MFA (idealmente passkey/FIDO2), sessão curta com reautenticação para ações sensíveis e log imutável de cada ação (quem, quando, o quê).

O que está acontecendo. requireAdmin compara o e-mail do usuário com a lista; nenhuma verificação adicional; nenhum `console.log` /registro nas ações. O painel só aparece na UI para admins, mas a autorização é feita no servidor (correto).

Por que isso é uma brecha. A conta administrativa é o 'cofre' do sistema: com ela, um invasor libera contas gratuitas indefinidamente (perda de receita) e, se o e-mail admin for descoberto (é o e-mail pessoal do dono), phishing ou reutilização de senha vazada bastam. Sem trilha, um abuso não é detectado nem reconstruído.

Sequência causal. Senha do admin obtida (phishing, vazamento, força bruta facilitada por AUD-003) → login normal → POST /api/admin/free-accounts → acesso gratuito para terceiros; nenhum registro para investigar.

Como alguém poderia abusar. Reutilização de senha do dono vazada em outro serviço → login → liberação de dezenas de e-mails gratuitos, vendidos por fora.

O que foi demonstrado. Não testado (não há MFA a testar; comportamento verificado por código e T09 confirma que usuários comuns recebem 403).

O que pode acontecer. Perda de receita, contas fantasmas, dificuldade de resposta a incidente.

Por que recebeu este nível. MÉDIA: exige comprometer a senha; impacto econômico e de governança, sem acesso a dados de clientes.

O que faria o nível subir ou descer. Sobe para ALTA se funções admin passarem a ler/exportar dados de usuários. Desce para BAIXA com MFA + log.

Como corrigir. 1) Exigir MFA (TOTP ou WebAuthn) para contas em ADMIN_EMAILS — ao menos step-up na primeira ação admin da sessão. 2) Registrar cada ação admin em coleção `AuditLog` (ator, alvo, ação, IP, data) e enviar e-mail ao próprio admin. 3) Reautenticação (senha) antes de adicionar/remover contas gratuitas. 4) Senha do admin com ≥12 caracteres e verificação contra senhas vazadas.

Como confirmar a correção. 1) Admin sem MFA configurado → painel exige cadastro do fator. 2) Adicionar conta gratuita → registro em AuditLog com o e-mail do admin. 3) Usuário comum → 403 (mantido).

Camada 2 — Detalhes técnicos

Evidência (redigida): admin.controller.ts e env.ts citados; T09 (403 para não-admin).

Risco residual: Phishing resistente exige WebAuthn; TOTP reduz mas não elimina.

AUD-009 — Campo avatarUrl aceito do cliente (mass assignment): URL arbitrária armazenada e usada na remoção de arquivos

- **Status:** CONFIRMADO (gravação) / PROVÁVEL (remoção de arquivo alheio) · **Confiança:** alta / média · **Severidade:** BAIXA
- **CVSS v4.0:** 3.5 — `CVSS:4.0/AV:N/AC:H/AT:P/PR:L/UI:N/VC:N/VI:L/VA:N/SC:N/SI:L/SA:N`
- **CWE:** CWE-915 Modificação de atributos controlada externamente (mass assignment); CWE-73 Controle externo de nome de arquivo · **OWASP Top 10:2025:** A01:2025 Broken Access Control · **API Top 10:2023:** API3:2023 Broken Object Property Level Authorization · **ASVS:** V4 API (propriedades) — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** `server/src/validators/client.validators.ts:17 ( avatarUrl: z.string().optional() em create/update); client.controller.ts deleteClient/uploadClientAvatar → utils/avatar.ts deleteAvatar ( del(avatarUrl, { token }) )`
- **Owner / prazo / esforço:** Desenvolvimento · MÉDIO, até 30 dias · baixo

Camada 1 — Entenda a brecha

O que encontramos. O usuário pode gravar qualquer texto no campo `avatarUrl` do seu cliente (deveria ser definido só pelo upload). Esse valor é usado depois para apagar o arquivo antigo no Vercel Blob com o token do servidor.

Como deveria funcionar. Campos derivados do servidor (URL do arquivo, ids, papéis, datas de pagamento) não devem ser aceitos do cliente; a remoção deve usar apenas caminhos que o servidor gerou.

O que está acontecendo. T08: PUT `/api/clients/:id` com `{"avatarUrl": "https://attacker.example/track.png"}` → 200 e valor armazenado. Em produção com `BLOB_READ_WRITE_TOKEN`, `deleteAvatar` chama `del(url, token)` para qualquer URL armazenada; a remoção de um blob de outro usuário exigiria conhecer sua URL (aleatória) — não demonstrado.

Por que isso é uma brecha. Aceitar o campo permite (a) apontar a foto para um servidor externo, que passa a receber o IP e o horário de cada visualização (rastreamento) e (b) pedir ao servidor que apague um arquivo cuja URL o atacante conheça, usando privilégio que o atacante não tem. Também permite armazenar strings sem limite (não há `max`).

Sequência causal. Cliente envia `avatarUrl` → validador aceita string qualquer → armazenado → `deleteClient/upload` chama `del(avatarUrl)` com o token do servidor → arquivo de terceiro pode ser apagado se a URL for conhecida.

Como alguém poderia abusar. Um usuário grava a URL de uma foto pública de outro usuário (obtida por compartilhamento) e remove o próprio cliente: o arquivo do outro usuário some.

O que foi demonstrado. Gravação de URL arbitrária (T08). Remoção cross-user não testada (produção/Blob fora do escopo).

O que pode acontecer. Rastreamento de acessos; remoção de fotos de outros usuários; poluição de dados.

Por que recebeu este nível. BAIXA: exige conhecer URLs aleatórias; o impacto é remoção de foto/rastreamento. Não INFORMATIVA porque há efeito real e privilégio do servidor envolvido.

O que faria o nível subir ou descer. Sobe para MÉDIA se URLs de blob forem listáveis ou previsíveis.

Como corrigir. 1) Remover `avatarUrl` dos schemas de create/update (ou aceitar apenas se igual ao valor atual). 2) Em `deleteAvatar`, só chamar `del` quando a URL começar com o prefixo do próprio store (`https://<store>.public.blob.vercel-storage.com/avatars/`) e contiver o `localId` do cliente. 3) Limitar tamanho de strings livres (`.max(...)`) em todos os validadores.

Como confirmar a correção. 1) PUT com avatarUrl externo → campo ignorado/400. 2) deleteAvatar com URL fora do prefixo → não chama `del`.

Camada 2 — Detalhes técnicos

Evidência (redigida): audit-evidence T08 linha 'stored avatarUrl = https://attacker.example/track.png'.

Risco residual: Nenhum relevante.

AUD-010 — Enumeração de contas: cadastro responde 'já cadastrado' e o login responde ~15× mais rápido para e-mails inexistentes

- **Status:** CONFIRMADO · **Confiança:** alta · **Severidade:** BAIXA
- **CVSS v4.0:** 2.3 — `CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N`
- **CWE:** CWE-204 Discrepância observável de resposta; CWE-208 Discrepância de tempo · **OWASP Top 10:2025:** A07:2025 Authentication Failures · **API Top 10:2023:** API2:2023 · **ASVS:** V6 Authentication (enumeração) — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** `server/src/controllers/auth.controller.ts requestRegisterCode (409 'Este e-mail já está cadastrado'); login ( if (!user) throw` antes de `comparePassword`)
- **Owner / prazo / esforço:** Desenvolvimento · MÉDIO, até 30 dias · baixo

Camada 1 — Entenda a brecha

O que encontramos. Dá para descobrir se um e-mail tem conta: o cadastro avisa explicitamente, e o login demora ~285 ms quando o e-mail existe (calcula o `bcrypt`) contra ~18 ms quando não existe.

Como deveria funcionar. Respostas e tempos devem ser indistinguíveis; no cadastro, enviar um e-mail informativo ao endereço já existente em vez de revelar na tela; no login, executar uma comparação de hash fictícia quando o usuário não existe.

O que está acontecendo. T07: 409 com mensagem explícita. T13: três medições — desconhecido 0,108/0,017/0,019 s; existente 0,360/0,286/0,286 s.

Por que isso é uma brecha. Saber quais e-mails têm conta alimenta credential stuffing (AUD-003), phishing direcionado e viola a privacidade dos clientes (revela que fulano usa o serviço). A recuperação de senha já é uniforme (controle validado), o que torna a falha localizada.

Sequência causal. E-mail alvo → POST request-code ou login → resposta/tempo diferente → confirmação de existência da conta.

Como alguém poderia abusar. Lista de e-mails de uma cidade → verificação em massa (limitada por AUD-003) → alvos para phishing de 'redefinição de senha'.

O que foi demonstrado. Ambos os canais.

O que pode acontecer. Privacidade e apoio a ataques de senha; sem acesso direto.

Por que recebeu este nível. BAIXA: informação limitada (existe/não existe) e usabilidade do cadastro conta a favor da mensagem explícita; corrigir o tempo do login é barato.

O que faria o nível subir ou descer. Sobe combinada com AUD-003/AUD-014.

Como corrigir. 1) Login: quando o usuário não existir, executar `comparePassword(data.password, DUMMY_HASH)` para igualar o tempo. 2) request-code: opcionalmente responder 200 genérico e enviar e-mail 'você já tem conta' ao endereço (trade-off de usabilidade — decisão de produto). 3) Manter a resposta uniforme do 'esqueci minha senha'.

Como confirmar a correção. Tempo médio de login para e-mail inexistente ≈ existente (±20%).

Camada 2 — Detalhes técnicos

Evidência (redigida): audit-evidence T07, T13.

Risco residual: Enumeração pelo cadastro permanece se a mensagem for mantida por usabilidade.

AUD-011 — Site (SPA) publicado sem cabeçalhos de segurança configurados — helmet protege apenas as respostas da API

- **Status:** PROVÁVEL · **Confiança:** média (produção não observada; configuração inexistente confirmada) · **Severidade:** BAIXA
- **CVSS v4.0:** 2.1 — CVSS:4.0/AV:N/AC:H/AT:N/PR:N/UI:A/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N
- **CWE:** CWE-1021 Restrição imprópria de frames (clickjacking); CWE-693 Falha de mecanismo de proteção · **OWASP Top 10:2025:** A02:2025 Security Misconfiguration · **API Top 10:2023:** API8:2023 · **ASVS:** V3 Web Frontend Security — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** vercel.json (sem bloco `headers`); client/index.html (2 scripts inline: tema e módulo); helmet só em server/src/app.ts (rotas /api e /uploads)
- **Owner / prazo / esforço:** Deploy · MÉDIO, até 30 dias · baixo

Camada 1 — Entenda a brecha

O que encontramos. As páginas do app são servidas como arquivos estáticos pela Vercel; o helmet, que envia CSP, X-Frame-Options e afins, roda só no Express (API). Não há configuração de cabeçalhos para o HTML/JS.

Como deveria funcionar. O HTML deve chegar com Content-Security-Policy (script-src restrito, frame-ancestors 'none'), X-Content-Type-Options, Referrer-Policy e Permissions-Policy; HSTS na borda.

O que está acontecendo. T01 mostra os cabeçalhos do helmet em /api/health; vercel.json não define `headers`. A Vercel aplica HSTS por padrão em domínios *.vercel.app (não verificado nesta auditoria). O React escapa saídas e não há `dangerouslySetInnerHTML` (verificado por grep), o que reduz o risco de XSS.

Por que isso é uma brecha. Sem `frame-ancestors`, o site pode ser embutido em um iframe invisível em outra página (clickjacking) para induzir cliques em 'Remover' ou 'Sair'. Sem CSP, um XSS futuro ou um script de terceiro comprometido (fontes do Google são carregadas por CSS) teria liberdade total. São defesas em profundidade, não uma falha explorável hoje.

Sequência causal. HTML sem CSP/frame-ancestors → página embutível em iframe ou scripts externos sem restrição → clickjacking / amplificação de XSS futuro.

Como alguém poderia abusar. Página maliciosa embute o app em iframe transparente sobre um botão 'clique para ganhar' alinhado ao 'Remover'; vítima logada clica.

O que foi demonstrado. Ausência de configuração; comportamento em produção não observado.

O que pode acontecer. Ações indesejadas por clickjacking; menor contenção de XSS.

Por que recebeu este nível. BAIXA: exige interação da vítima e não há XSS conhecido; correção de configuração simples.

O que faria o nível subir ou descer. Sobe se surgir XSS ou se scripts de terceiros forem adicionados.

Como corrigir. Adicionar em vercel.json: `"headers": [{ "source": "/(.*)", "headers": [{ "key": "X-Frame-Options", "value": "DENY" }, { "key": "X-Content-Type-Options", "value": "nosniff" }, { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" }, { "key": "Permissions-Policy", "value": "camera=(), microphone=(), geolocation=()" }, { "key": "Content-Security-Policy", "value": "default-src 'self'; script-src 'self' 'sha256-'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src https://fonts.gstatic.com; img-src 'self' data: https://*.public.blob.vercel-storage.com; c"`

`connect-src 'self' https://api.mercadopago.com; frame-ancestors 'none'; base-uri 'self'; form-action 'self' }`. Mover o script inline do tema para arquivo próprio para dispensar o hash.

Como confirmar a correção. curl -I na home em produção → cabeçalhos presentes; app carrega sem erros de CSP no console; fontes e avatares carregam.

Camada 2 — Detalhes técnicos

Evidência (redigida): vercel.json (16 linhas, sem headers); T01.

Risco residual: CSP com 'unsafe-inline' em style permanece por causa do Tailwind/estilos inline.

AUD-012 — CORS libera permanentemente as origens localhost/capacitor com credenciais, inclusive em produção

- **Status:** CONFIRMADO · **Confiança:** alta · **Severidade:** BAIXA
- **CVSS v4.0:** 2.0 — `CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:A/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N`
- **CWE:** CWE-942 Política CORS permissiva · **OWASP Top 10:2025:** A02:2025 Security Misconfiguration · **API Top 10:2023:** API8:2023 · **ASVS:** V3 / V4 (CORS) — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** server/src/config/env.ts `clientOrigins .concat(['https://localhost', 'capacitor://localhost', 'http://localhost'])`; app.ts `middleware CORS`
- **Owner / prazo / esforço:** Desenvolvimento · MÉDIO, até 30 dias · baixo

Camada 1 — Entenda a brecha

O que encontramos. Qualquer página servida em `http://localhost` ou `https://localhost` na máquina do usuário recebe permissão de chamar a API com credenciais (`Access-Control-Allow-Credentials: true`).

Como deveria funcionar. Em produção, apenas as origens do site (e do app nativo, se ainda existir) devem constar na allowlist; as origens de desenvolvimento devem depender de `NODE_ENV`.

O que está acontecendo. T02b: `Origin: http://localhost` → `Access-Control-Allow-Origin: http://localhost` + credenciais. Origem estranha (`evil.example`) não recebe cabeçalhos (controle validado). O cookie é `SameSite=Lax`, o que impede o navegador de anexá-lo em requisições cross-site iniciadas por script, limitando bastante o abuso.

Por que isso é uma brecha. Se um software local malicioso (ou um servidor de desenvolvimento de outro projeto na porta 80/443) abrir uma página em localhost, o navegador permitiria ler respostas da API — mas só se o cookie fosse enviado, o que `SameSite=Lax` bloqueia. O risco real hoje é residual; fica registrado porque o app Android (Capacitor) foi descontinuado e a exceção perdeu a razão de existir.

Sequência causal. Página em localhost → fetch com credentials → ACAO reflete localhost → (bloqueado por `SameSite=Lax` no cookie) → sem impacto atual.

Como alguém poderia abusar. Cenário limitado a navegadores antigos sem `SameSite` padrão ou a mudanças futuras de cookie.

O que foi demonstrado. Cabeçalhos CORS refletidos (T02b).

O que pode acontecer. Leitura de dados da vítima a partir de página local, apenas se o cookie deixar de ser `Lax`.

Por que recebeu este nível. BAIXA: bloqueado na prática por `SameSite`; é limpeza de superfície.

O que faria o nível subir ou descer. Sobe para MÉDIA se o cookie mudar para `SameSite=None`.

Como corrigir. Incluir as origens de desenvolvimento apenas quando `!env.isProduction`; remover `capacitor://localhost` / `https://localhost` já que o APK foi retirado.

Como confirmar a correção. Em produção, `Origin: http://localhost` → sem ACAO.

Camada 2 — Detalhes técnicos

Evidência (redigida): audit-evidence T02/T02b.

Risco residual: Nenhum.

AUD-013 — Proteção contra CSRF depende exclusivamente de SameSite=Lax (sem token nem checagem de Origin nas mutações)

- **Status:** CONFIRMADO · **Confiança:** alta · **Severidade:** BAIXA
- **CVSS v4.0:** 2.3 — CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:A/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N
- **CWE:** CWE-352 Cross-Site Request Forgery · **OWASP Top 10:2025:** A01:2025 Broken Access Control · **API Top 10:2023:** API8:2023 · **ASVS:** V4 API and Web Service (CSRF) — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** server/src/controllers/auth.controller.ts setAuthCookie (sameSite: 'lax'); app.ts (sem verificação de Origin em POST/PUT/DELETE; express.json aceita só application/json; multer em /clients/:id/avatar aceita multipart)
- **Owner / prazo / esforço:** Desenvolvimento · ESTRATÉGICO, até 90 dias · baixo

Camada 1 — Entenda a brecha

O que encontramos. Nada além do atributo SameSite=Lax do cookie impede que um site externo faça o navegador da vítima enviar uma requisição de alteração para a API.

Como deveria funcionar. Defesa em profundidade: SameSite + validação do cabeçalho Origin/Referer nas mutações (ou token anti-CSRF), e rejeição de content-types não esperados.

O que está acontecendo. Requisições JSON não podem ser criadas por formulários HTML e `express.json` ignora `text/plain`, o que protege as rotas JSON. O upload multipart poderia ser disparado por um `<form>` cross-site, mas o navegador não envia o cookie Lax em POST cross-site — logo, hoje nenhum caminho foi demonstrado.

Por que isso é uma brecha. CSRF ('falsificação de requisição entre sites') é quando um site malicioso usa a sessão da vítima para agir em seu nome. A proteção atual é correta para navegadores modernos, mas única: uma futura mudança para SameSite=None (ex.: para embutir o app) abriria todas as mutações de uma vez.

Sequência causal. Página maliciosa → formulário POST multipart para `/api/clients/:id/avatar` → cookie não enviado (Lax) → 401. Se Lax fosse removido → upload no nome da vítima.

Como alguém poderia abusar. Não aplicável hoje; registrado como defesa em profundidade.

O que foi demonstrado. Análise de código; nenhum caminho de exploração atual.

O que pode acontecer. Alterações na conta da vítima caso a única camada caia.

Por que recebeu este nível. BAIXA: controle existente e eficaz; achado de arquitetura.

O que faria o nível subir ou descer. Sobe para ALTA se SameSite for relaxado.

Como corrigir. Middleware nas rotas de mutação: rejeitar quando `Origin / Referer` presente e fora da allowlist (o app nativo com Bearer não envia cookie e pode ser isento); usar prefixo `__Host-` no nome do cookie em produção; exigir `Content-Type: application/json` nas rotas JSON.

Como confirmar a correção. POST `/api/clients` com Origin externo e cookie válido → 403; com Origin do site → 201.

Camada 2 — Detalhes técnicos

Evidência (redigida): auth.controller.ts setAuthCookie; app.ts.

Risco residual: Nenhum.

AUD-014 — Política de senha abaixo do recomendado (mínimo de 6 caracteres, sem bloqueio de senhas vazadas)

- **Status:** CONFIRMADO · **Confiança:** alta · **Severidade:** BAIXA
- **CVSS v4.0:** 2.7 — `CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N`
- **CWE:** CWE-521 Requisitos fracos de senha · **OWASP Top 10:2025:** A07:2025 Authentication Failures · **API Top 10:2023:** API2:2023 · **ASVS:** V6 Authentication (senhas) — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** `server/src/validators/auth.validators.ts newPasswordField min(6)` ; `client/src/pages/Register.tsx` e `ResetPassword.tsx` (mesmo mínimo)
- **Owner / prazo / esforço:** Desenvolvimento · MÉDIO, até 30 dias · baixo

Camada 1 — Entenda a brecha

O que encontramos. Senhas de 6 caracteres são aceitas e não há verificação contra listas de senhas já vazadas (ex.: '123456' passa).

Como deveria funcionar. NIST SP 800-63B: mínimo de 8 caracteres (15 quando sem MFA é recomendado), comparação com senhas comprometidas/comuns, sem regras de composição arbitrárias, permitir senhas longas.

O que está acontecendo. `z.string().min(6)` ; nenhum blocklist; bcrypt custo 12 (adequado — controle validado).

Por que isso é uma brecha. Com AUD-003 (limite por IP quebrado) e AUD-010 (enumeração), senhas curtas e comuns são o caminho mais barato para invadir contas; o hash forte só protege se a senha não for trivial.

Sequência causal. Senha '123456' aceita → e-mail conhecido → tentativas de senhas comuns (limitadas fracamente) → acesso.

Como alguém poderia abusar. Credential stuffing com pares e-mail/senha vazados de outros serviços.

O que foi demonstrado. Regra de validação lida no código.

O que pode acontecer. Takeover de contas com senhas fracas.

Por que recebeu este nível. BAIXA isoladamente; relevante em cadeia com AUD-003.

O que faria o nível subir ou descer. Sobe para MÉDIA enquanto AUD-003 não for corrigida.

Como corrigir. `min(8)` (idealmente 10) + `max(128)` ; rejeitar as ~10 mil senhas mais comuns (lista local) ou consultar a API k-anonymity do Have I Been Pwned (prefixo SHA-1 de 5 caracteres, sem enviar a senha); indicador de força no formulário.

Como confirmar a correção. Cadastro com '123456' → 400; com 'correto cavalo bateria' → 201.

Camada 2 — Detalhes técnicos

Evidência (redigida): `auth.validators.ts` linha do `newPasswordField`.

Risco residual: Reutilização de senhas fortes vazadas em outros sites só é mitigada por MFA.

AUD-015 — Webhook de pagamento aceita chamadas anônimas sem assinatura (segredo opcional) e sem limite de taxa

- **Status:** CONFIRMADO · **Confiança:** alta · **Severidade:** BAIXA
- **CVSS v4.0:** 2.0 — `CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N`

- **CWE:** CWE-306 Ausência de autenticação para função crítica (parcial); CWE-770 · **OWASP Top 10:2025:** A02:2025 Security Misconfiguration · **API Top 10:2023:** API4:2023 / API10:2023 Unsafe Consumption of APIs · **ASVS:** V4 (webhooks) — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** server/src/app.ts (POST /api/billing/webhook); services/billing.ts verifyWebhookSignature (`if (!secret) return true`)
- **Owner / prazo / esforço:** Deploy / Desenvolvimento · CURTO, até 7 dias (antes de ativar a cobrança) · baixo

Camada 1 — Entenda a brecha

O que encontramos. Se MP_WEBHOOK_SECRET não for definido, qualquer pessoa pode chamar o webhook com um id de pagamento; o servidor então consulta a API do Mercado Pago com o token da loja. Não há limite de chamadas.

Como deveria funcionar. Webhooks devem exigir assinatura válida (obrigatória em produção), ter limite de taxa e registrar tentativas inválidas.

O que está acontecendo. T11: POST anônimo com corpo arbitrário → 200. Ponto positivo (controle validado): o servidor nunca confia no corpo — busca o pagamento no Mercado Pago e só aplica se aprovado e vinculado ao usuário pelo `external_reference` ; por isso não há fraude de crédito por esse caminho.

Por que isso é uma brecha. Sem assinatura, o endpoint vira um 'amplificador': cada chamada anônima gera uma chamada autenticada ao Mercado Pago, podendo esgotar cotas ou gerar bloqueio temporário da integração (indisponibilidade dos pagamentos). A assinatura também é a única forma de saber que a notificação veio do Mercado Pago.

Sequência causal. POST anônimo em massa → aplicação consulta MP a cada chamada → cota/limite da API do MP consumida → confirmações legítimas falham.

Como alguém poderia abusar. Script chama o webhook milhares de vezes com ids aleatórios.

O que foi demonstrado. Aceitação de chamada anônima (T11).

O que pode acontecer. Indisponibilidade da confirmação de pagamentos; custo de logs.

Por que recebeu este nível. BAIXA: sem impacto de integridade (verificado no código); apenas consumo.

O que faria o nível subir ou descer. Sobe para MÉDIA se a lógica passar a confiar em campos do corpo.

Como corrigir. 1) Tornar MP_WEBHOOK_SECRET obrigatório em produção (falhar no boot). 2) Rate limit no webhook e no /billing/confirm. 3) Validar formato numérico do id antes de consultar. 4) Logar assinaturas inválidas com IP.

Como confirmar a correção. Webhook sem assinatura em produção → 401; com assinatura correta → 200.

Camada 2 — Detalhes técnicos

Evidência (redigida): audit-evidence T11; billing.ts verifyWebhookSignature.

Risco residual: Nenhum.

AUD-016 — Dependências com vulnerabilidades conhecidas (npm audit: 4 altas, 8 moderadas) — alcance avaliado por pacote

- **Status:** CONFIRMADO (versões) / alcance por pacote no texto · **Confiança:** alta · **Severidade:** BAIXA (exceto sharp/multer, tratados em AUD-002)
- **CVSS v4.0:** 3.7 — `CVSS:4.0/AV:N/AC:L/AT:P/PR:L/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N`

- **CWE:** CWE-1395 Dependência vulnerável · **OWASP Top 10:2025:** A03:2025 Software Supply Chain Failures · **API Top 10:2023:** API8:2023 · **ASVS:** V15 Secure Coding and Architecture (componentes) — referência exata não verificada
- **CISA KEV / EPSS:** não consultado (nenhum CVE com KEV identificado nos avisos listados)
- **Onde está:** package-lock.json (raiz do workspace); server/package.json; client/package.json
- **Owner / prazo / esforço:** Desenvolvimento · MÉDIO, até 30 dias · baixo

Camada 1 — Entenda a brecha

O que encontramos. O `npm audit` aponta 12 avisos. Após análise de alcance: nodemailer 6.10.1 (avisos de CRLF/DoS no parser de endereços — entradas aqui passam por validação `email()` do zod e não usam `raw` / `jsonTransport`, alcance baixo); express 4.22.2/body-parser/qs (bypass de limite de array e DoS em `isBuffer`, moderado, alcançável via query string); mercadopago→uuid 9 (bounds check em v3/v5 com buffer — não usado nesse modo, alcance improvável); morgan (log forging — só em desenvolvimento); @capacitor/cli→xmldom/xcode (somente build local do APK, descontinuado); sharp e multer (alcançáveis — AUD-002).

Como deveria funcionar. Dependências atualizadas rotineiramente, `npm audit` no build e remoção do que não é mais usado.

O que está acontecendo. Lista completa em audit-evidence e no JSON deste achado. Lockfile presente (controle validado); Vercel instala com `npm install` a partir dele.

Por que isso é uma brecha. Bibliotecas de terceiros executam com os mesmos privilégios do servidor. Mesmo avisos 'moderados' em parsing de query string (qs) podem virar negação de serviço com uma requisição bem construída.

Sequência causal. Versão vulnerável instalada → entrada externa alcança a função afetada (query string, multipart, imagem) → DoS ou pior.

Como alguém poderia abusar. Requisição com query string patológica para qs; ver AUD-002 para upload.

O que foi demonstrado. Versões e avisos; alcance por leitura de código.

O que pode acontecer. Negação de serviço; comprometimento via decodificadores (AUD-002).

Por que recebeu este nível. BAIXA para o conjunto residual (fora sharp/multer) por alcance limitado; correção é `npm update`.

O que faria o nível subir ou descer. Sobe conforme novos avisos; consultar KEV/EPSS ao atualizar.

Como corrigir. 1) `npm update` nos pacotes diretos (sharp, multer, nodemailer, morgan, express); `npm audit fix`. 2) Remover @capacitor/* e o diretório client/android se o app nativo não voltar. 3) Adicionar `npm audit --audit-level=high` ao script de build/CI e dependabot/renovate no GitHub.

Como confirmar a correção. `npm audit --audit-level=high` → 0 vulnerabilidades.

Camada 2 — Detalhes técnicos

Evidência (redigida): `npm audit --json` (resumo: high 4, moderate 8) executado em 2026-09-14; `npm ls` para caminhos transitivos.

Risco residual: Avisos futuros; processo contínuo.

AUD-017 — Sem registro de eventos de segurança nem alertas (logins falhos, redefinições, ações admin, pagamentos)

- **Status:** CONFIRMADO · **Confiança:** alta · **Severidade:** BAIXA
- **CVSS v4.0:** 0.0 — não aplicável (ausência de controle detectivo)

- **CWE:** CWE-778 Registro insuficiente; CWE-223 Omissão de informação relevante para segurança · **OWASP Top 10:2025:** A09:2025 Security Logging and Alerting Failures · **API Top 10:2023:** API8:2023 · **ASVS:** V16 Security Logging and Error Handling — referência exata não verificada
- **CISA KEV / EPSS:** não aplicável
- **Onde está:** server/src/app.ts (morgan só fora de produção); controllers de auth/admin/billing (apenas console.error em falhas)
- **Owner / prazo / esforço:** Desenvolvimento / Operação · MÉDIO, até 30 dias · médio

Camada 1 — Entenda a brecha

O que encontramos. Em produção o servidor não registra quem tentou entrar, quem redefiniu senha, quem liberou contas gratuitas ou quais pagamentos foram aplicados; erros vão para o log efêmero da Vercel.

Como deveria funcionar. Eventos mínimos com ator, tenant, ação, resultado, IP e data, enviados a um destino com retenção e alertas para anomalias (rajadas de falhas, ações admin).

O que está acontecendo. Nenhuma chamada de log estruturado nos fluxos citados; `morgan('dev')` desativado em produção.

Por que isso é uma brecha. Sem registros não há como perceber um ataque de senha em andamento, provar quem liberou uma conta ou reconstruir um incidente — os achados AUD-001/003/008 ficariam invisíveis mesmo em exploração.

Sequência causal. Ataque ou abuso → nenhum evento gravado → nenhuma detecção → descoberta apenas por efeito colateral (reclamação, fatura).

Como alguém poderia abusar. Não é um vetor por si; é a ausência de resposta.

O que foi demonstrado. Leitura de código.

O que pode acontecer. Incidentes prolongados e não investigáveis.

Por que recebeu este nível. BAIXA como item isolado; multiplicador de impacto dos demais.

O que faria o nível subir ou descer. —

Como corrigir. 1) Logger estruturado (pino) com eventos: login ok/falha (sem senha), reset solicitado/concluído, troca de senha, admin.free_account.add/remove, billing.payment.applied, webhook inválido, 402/403. 2) Enviar para Vercel Log Drain ou serviço (Axiom/Logtail) com retenção ≥90 dias. 3) Alertas: > 10 falhas de login por conta/hora; qualquer ação admin; erro 5xx acima de limiar. 4) Nunca registrar tokens, senhas ou corpo de e-mail.

Como confirmar a correção. Simular 3 logins falhos → 3 eventos no destino com ator e IP; ação admin → evento com e-mail do admin.

Camada 2 — Detalhes técnicos

Evidência (redigida): app.ts linhas do morgan; grep por `console.log` /logger nos controllers.

Risco residual: Depende de alguém olhar os alertas; definir responsável.

AUD-018 — Observações informativas: token no corpo da resposta web, usuário/hosts do Atlas no repositório, erro 413 mapeado como 500, dados locais no dispositivo, backup do Atlas não verificado

- **Status:** INFORMATIVO · **Confiança:** alta · **Severidade:** INFORMATIVA
- **CVSS v4.0:** 0.0 — não aplicável
- **CWE:** CWE-200 (exposição de informação de configuração); CWE-209 (mensagens de erro) · **OWASP Top 10:2025:** A02:2025 Security Misconfiguration · **API Top 10:2023:** API8:2023 · **ASVS:** V13 Configuration / V14 Data Protection — referência exata não verificada

- **CISA KEV / EPSS:** não aplicável
- **Onde está:** auth.controller.ts login/register (token no JSON também para web); server/.env.example (usuário ..._db_user e hosts do cluster, sem senha); errorHandler.ts (entity.too.large → 500, T14); client/src/db (IndexedDB com dados do usuário até logout); MongoDB Atlas (plano/backup desconhecido)
- **Owner / prazo / esforço:** Desenvolvimento / Operação · ESTRATÉGICO, até 90 dias (exceto (e): verificar em 7 dias) · baixo

Camada 1 — Entenda a brecha

O que encontramos. Conjunto de pontos sem caminho de exploração demonstrável: (a) o JWT é devolvido no corpo do login mesmo para o site, onde o cookie já basta — fica visível a extensões/scripts; (b) o .env.example público revela o nome do usuário do banco e os hosts do Atlas (a senha está mascarada); (c) corpo acima de 100 kB responde 500 em vez de 413; (d) os dados ficam no IndexedDB do navegador até 'Sair' — em computadores compartilhados, fechar a aba não os remove; (e) não foi possível verificar se o cluster tem backup e teste de restauração (clusters gratuitos M0 não têm backup automático).

Como deveria funcionar. Minimizar o que sai na resposta; exemplos de configuração sem dados reais; códigos HTTP corretos; aviso ao usuário sobre dispositivos compartilhados; backup verificado.

O que está acontecendo. Itens listados verificados por código/teste (T14) e leitura do repositório; backup não verificável.

Por que isso é uma brecha. Cada item reduz a margem de erro em cenários futuros (extensão maliciosa, ataque ao Atlas com usuário conhecido, perda de dados sem backup).

Sequência causal. —

Como alguém poderia abusar. —

O que foi demonstrado. Ver 'observed'.

O que pode acontecer. Baixo; item (e) pode significar perda total de dados em falha do banco.

Por que recebeu este nível. INFORMATIVA por definição do prompt: sem exploração demonstrável.

O que faria o nível subir ou descer. (e) vira ALTA se confirmado que não há backup.

Como corrigir. (a) Devolver token só quando a requisição vier com header do app nativo (ou removê-lo, já que o APK foi descontinuado). (b) Trocar o exemplo por mongodb+srv://USUARIO:SENHA@SEU-CLUSTER/gestor. (c) Mapear entity.too.large → 413. (d) Texto em Configurações: 'Em computador compartilhado, clique em Sair'. (e) Confirmar plano do Atlas; em M0, agendar mongodump semanal para storage privado e testar restauração.

Como confirmar a correção. —

Camada 2 — Detalhes técnicos

Evidência (redigida): T14 (500 para 200 kB); .env.example linha MONGO_URL; auth.controller.ts.

Risco residual: —

05 — Controles positivos comprovados

Somente controles com evidência de teste ou de código. "Não encontrei" não foi tratado como aprovação.

ID	Controle	Evidência	Alcance
AU D- CV- 01	Isolamento por usuário em todos os dados (BOLA/IDOR ausente)	Teste T08: usuário canário tentou GET/PUT/DELETE de cliente de outro usuário → 404 em todos; lista do canário vazia; anônimo → 401. Código: todas as consultas de clients/tasks/finance/goals incluem <code>userId: req.userId</code> (grep verificado).	Alto — cobre leitura, escrita, exclusão e listagem.
AU D- CV- 02	Autenticação obrigatória e JWT <code>alg: none</code> rejeitado	T04: token com <code>alg=none</code> → 401. <code>requireAuth</code> exige cookie ou Bearer válido.	Alto
AU D- CV- 03	Cookie de sessão <code>HttpOnly</code> + <code>SameSite=Lax</code> (+ <code>Secure</code> em produção pelo código)	T03: Set-Cookie com <code>HttpOnly</code> ; <code>SameSite=Lax</code> ; <code>secure: env.isProduction</code> em <code>setAuthCookie</code> .	Médio — ver AUD-013 (única camada CSRF).
AU D- CV- 04	Hash de senha com <code>bcrypt</code> custo 12	<code>server/src/utils/password.ts</code> ; T13 mostra ~280 ms por comparação.	Alto
AU D- CV- 05	Tokens de recuperação e códigos de cadastro: aleatórios, armazenados como HMAC, uso único, expiração curta, tentativas limitadas, cooldown	<code>services/verification.ts</code> ; T07 comportamento de cooldown observado em sessão anterior (429 'Aguarde 60s').	Alto
AU D- CV- 06	Resposta uniforme no 'esqueci minha senha'	<code>forgotPassword</code> devolve a mesma mensagem existindo ou não a conta (código).	Alto
AU D- CV- 07	Validação de entrada com <code>zod</code> em todas as rotas (tipos estritos impedem injeção NoSQL por objetos)	Validators por módulo; <code>mongoose.set('strictQuery', true)</code> ; e-mails validados com <code>.email()</code> .	Alto (exceto regex de busca — AUD-006)
AU D- CV- 08	Erros genéricos sem stack trace; JSON malformatado → 400	T06.	Alto
AU D- CV- 09	Nenhum segredo no histórico do Git	Busca por padrões (<code>mongodb://user:pass@</code> , <code>JWT_SECRET=valor</code> , <code>APP_USR=</code> , <code>SMTP_PASS=valor</code> , chaves privadas) em <code>git log --all -p</code> : apenas placeholders em <code>.env.example/README</code> . <code>.env</code> ignorado (git check-ignore).	Médio — busca por padrões, não exaustiva.

ID	Controle	Evidência	Alcance
AU D- CV- 10	Frontend sem sinks perigosos e sem source maps publicados	grep: nenhum dangerouslySetInnerHTML/innerHTML/eval; dist sem .map; nenhum token em localStorage no web.	Alto
AU D- CV- 11	helmet ativo na API (CSP, HSTS, nosniff, X-Frame-Options, Referrer-Policy)	T01.	Médio — não cobre o SPA (AUD-011).
AU D- CV- 12	CORS rejeita origens desconhecidas	T02: Origin: https://evil.example → sem cabeçalhos ACAA.	Alto (ressalva AUD-012)
AU D- CV- 13	Autorização de função administrativa no servidor	T09: usuário comum → 403 em /api/admin/free-accounts.	Alto
AU D- CV- 14	Bloqueio de plano no servidor (402) e não só na interface	T10: canário com teste expirado → 402 em /api/clients; /billing/status reporta allowed: false .	Alto
AU D- CV- 15	Pagamento: preço definido no servidor, idempotência por id do pagamento e vínculo ao usuário via external_reference; corpo do webhook nunca é confiável	billing.ts createCheckout/applyPaymentById; confirm verifica result.userId !== String(user._id) → 403.	Médio — ver AUD-007.
AU D- CV- 16	Imagens de avatar reprocessadas (resize + WebP) — SVG/HTML/polyglots não são servidos como enviados	utils/avatar.ts processAvatar; nome de arquivo gerado pelo servidor; path.basename na remoção local.	Médio — ver AUD-002.
AU D- CV- 17	E-mails com escape de HTML nos campos do usuário	services/mail.ts escapeHtml aplicado a nome e link.	Alto
AU D- CV- 18	Limite de corpo JSON (100 kB padrão do express.json) e limite de tamanho de upload (5 MB)	T14 (rejeitado, embora com 500); uploadAvatar.ts limits.fileSize.	Médio

06 — Roteiro de correção (ordenado por redução de risco)

Nenhuma correção foi aplicada (APPLY_FIXES=false). Esforço estimado para um desenvolvedor familiarizado com o projeto.

URGENTE — 24 a 48 h

#	Ação	Achado	Esforço	Teste de aceitação
1	Apagar <code>demo@gestorpro.com</code> do banco de produção (ou trocar a senha por uma gerada e não publicada); em <code>server/src/seed.ts</code> , abortar quando <code>NODE_ENV=production</code> e gerar senha aleatória impressa no console	AUD-001	30 min	login com a senha do arquivo → 401
2	Definir <code>APP_URL</code> na Vercel e torná-lo obrigatório em produção (<code>required('APP_URL')</code> quando <code>isProduction</code>); remover o ramo que aceita <code>Origin</code> genérico em <code>resolveAppUrl</code>	AUD-005	30 min	link no e-mail de reset começa com <code>APP_URL</code> mesmo com Host/Origin forjados
3	Tornar <code>MP_WEBHOOK_SECRET</code> obrigatório em produção; cadastrar o webhook no painel do Mercado Pago	AUD-015	30 min	webhook sem assinatura → 401

CURTO — até 7 dias (antes de ativar a cobrança)

#	Ação	Achado	Esforço	Teste de aceitação
4	<code>npm install sharp@^0.35.4 multer@latest -w server</code> ; em <code>avatar.ts</code> conferir <code>metadata().format ∈ {jpeg,png,webp}</code> antes de processar; <code>limitInputPixels</code> ; <code>limits.fields/parts</code> no multer	AUD-002 / AUD-016	1 h	AVIF com MIME <code>image/png</code> → 400; <code>npm audit --audit-level=high</code> limpo em sharp/multer
5	<code>app.set('trust proxy', 1)</code> ; store compartilhado para express-rate-limit (Mongo/Redis); limite adicional por conta em login e request-code; <code>Retry-After</code>	AUD-003	3 h	XFF distintos não se somam; 10 falhas na mesma conta → 429
6	<code>applyPaymentById</code> atômico (<code>findOneAndUpdate({providerPaymentId, appliedAt:null, status:'approved'}, {\$set:{appliedAt}})</code>), validar <code>transaction_amount / currency_id</code> , tratar <code>refunded / charged _back</code> ; rate limit em <code>/billing/confirm</code> e webhook	AUD-007 / AUD-015	3 h	dois confirms concorrentes → +30 dias uma vez; estorno reduz <code>paidUntil</code>

MÉDIO — até 30 dias

#	Ação	Achado	Esforço	Teste de aceitação
7	<code>sessionVersion</code> no usuário e no JWT; incrementar em reset/troca de senha e "sair de todos"; <code>jwt.verify(..., {algorithms:['HS256']})</code> ; validade 7 d → curta com renovação, ou inatividade 24 h	AUD-004	4 h	token anterior ao reset → 401
8	Escapar regex e <code>max(100)</code> no <code>search</code> dos três controllers; <code>SyntaxError</code> → 400	AUD-006	1 h	<code>?search=(</code> → 200/400, nunca 500
9	MFA (TOTP/WebAuthn) para <code>ADMIN_EMAILS</code> ; coleção <code>AuditLog</code> para ações admin; reautenticação antes de liberar contas	AUD-008	1–2 dias	ação admin gera registro; painel exige fator
10	Remover <code>avatarUrl</code> dos schemas de create/update; <code>deleteAvatar</code> só apaga URLs do próprio store/ <code>localId</code> ; <code>max()</code> em strings livres	AUD-009	1 h	PUT com <code>avatarUrl</code> externo → ignorado
11	<code>comparePassword</code> contra hash fictício quando o usuário não existe; decidir sobre a mensagem 409 do cadastro	AUD-010	30 min	tempos equivalentes
12	Bloco <code>headers</code> no <code>vercel.json</code> (CSP com hash do script inline ou script externo, <code>frame-ancestors 'none'</code> , nosniff, Referrer-Policy, Permissions-Policy)	AUD-011	2 h	<code>curl -I</code> na home mostra cabeçalhos; app sem erros de CSP
13	<code>min(8) / max(128)</code> + blocklist de senhas comuns (lista local ou HIBP k-anonymity)	AUD-014	2 h	<code>123456</code> → 400
14	<code>npm audit --audit-level=high</code> no build; dependabot; remover <code>@capacitor/*</code> e <code>client/android</code> se o nativo não voltar	AUD-016	1 h	build falha com vulnerabilidade alta
15	Logger estruturado (pino) para eventos de segurança + log drain com retenção ≥ 90 dias + alertas básicos	AUD-017	1 dia	eventos visíveis no destino
16	Confirmar plano/backup do Atlas; em M0, <code>mongodump</code> agendado para storage privado e teste de restauração	AUD-018e	2 h	restauração em cluster de teste bem-sucedida

ESTRATÉGICO — até 90 dias

#	Ação	Achado	Esforço
17	Verificação de <code>Origin</code> / <code>Referer</code> nas mutações; cookie <code>__Host-token</code> em produção; exigir <code>Content-Type: application/json</code>	AUD-013	2 h
18	Origens de desenvolvimento no CORS apenas fora de produção	AUD-012	15 min
19	<code>token</code> no corpo do login só para o app nativo (ou remover); <code>.env.example</code> sem usuário/hosts reais; <code>entity.too.large</code> → 413; aviso sobre dispositivos compartilhados	AUD-018	1 h
20	SBOM (CycloneDX via <code>npm sbom</code>), plano de resposta a incidentes (rotação de <code>JWT_SECRET</code> , <code>MP_ACCE</code> <code>SS_TOKEN</code> , <code>SMTP_PASS</code>), <code>security.txt</code>	—	1 dia

Dependências entre itens

- 5 (trust proxy/limites) potencializa 11 e 13; sem 5, enumeração + senhas fracas continuam exploráveis.
- 7 (revogação) deve preceder qualquer rotação planejada de `JWT_SECRET`.
- 6 e 3 devem estar prontos antes de `MP_ACCESS_TOKEN` entrar em produção.

Risco residual esperado após o roteiro

Sem falhas ALTAS conhecidas; residual MÉDIO em autenticação por fator único para usuários comuns (MFA opcional é uma evolução), e infraestrutura (Vercel/Atlas) dependente de configuração não auditável por código.

07 — Limitações e itens não testados

Registro honesto do que esta auditoria **não** conseguiu comprovar. Cada item reduz a confiança da conclusão correspondente.

Ativos sem acesso

Ativo	O que não foi verificado	Impacto na confiança
Site publica do na Vercel	Cabeçalhos HTTP reais (HSTS, CSP), redirecionamento HTTPS, variáveis de ambiente efetivas (<code>NODE_ENV</code> , <code>APP_URL</code> , <code>CLIENT_ORIGIN</code> , <code>MP_*</code> , <code>ADMIN_EMAILS</code>), comportamento do proxy com <code>X-Forwarded-*</code> , logs	AUD-005, AUD-011, AUD-012 permanecem PROVÁVEL/condicionados; AUD-003 confirmado por código + teste local, mas o efeito exato na Vercel (instâncias) é inferido
Mongo DB Atlas	Lista de IPs permitidos, usuário/permissões do banco, criptografia em repouso, backups e teste de restauração , plano (M0?)	AUD-018e é hipótese; perda de dados por falha não coberta
Vercel Blob	ACL do store, listagem, capacidade de <code>del</code> em URLs de outros usuários	AUD-009 (parte de remoção) é PROVÁVEL
Mercado Pago	Fluxo real de pagamento, webhook autêntico, concorrência webhook/confirm, valores	AUD-007 é PROVÁVEL (código)
Provedores de e-mail	Entrega, SPF/DKIM/DMARC do domínio remetente (phishing com o nome do produto)	não avaliado
GitHub	Proteção de branch, revisores, dependabot, secret scanning, tokens de deploy	A08 (integridade de build/CI) NÃO TESTADO

Testes dinâmicos deliberadamente não executados

- **ReDoS real** contra o banco (AUD-006): padrão patológico não enviado porque o servidor local usa o cluster de produção compartilhado.
- **Exploit de imagem** (AUD-002): nenhum arquivo malicioso foi construído/enviado.
- **Envio de e-mails** (request-code, forgot-password com Host forjado): o `.env` local tem SMTP real; evitado para não gerar e-mails e custo.
- **Pagamentos**: nenhum checkout criado com token real.
- **Força bruta/credential stuffing**: apenas 22 requisições de login com senha errada para medir o limitador; nenhuma tentativa de adivinhar senhas reais.
- **Carga/DoS**: nenhum.

Contas e papéis

- Não existiam contas de teste com papéis diferentes; a auditoria criou e removeu um usuário canário comum. Não foi testada uma conta admin real (o painel admin foi validado por código e por 403 para usuário comum; o fluxo admin completo foi exercitado em sessão de desenvolvimento anterior com conta temporária, não como parte formal desta auditoria).
- Não há tenant além do usuário individual; testes cross-tenant equivalem a cross-usuário (executados).

Ferramentas

- Sem Semgrep/CodeQL (SAST automatizado), Gitleaks (segredos), OSV-Scanner/Trivy (SCA/SBOM), ZAP (DAST). Nada foi instalado, conforme a Regra Zero. Cobertura de SAST/segredos é manual e dirigida: pode haver padrões não cobertos.
- `npm audit` consultou o registro npm com os nomes/versões dos pacotes (metadados públicos), sem enviar código.
- KEV e EPSS não consultados online.

Requisitos não aplicáveis

ASVS V10 (OAuth/OIDC), V17 (WebRTC), OWASP AISVS/LLMSVS (sem IA), MASVS (app móvel não distribuído — `flutter_app/` e `client/android` fora do escopo). O app Flutter chama os mesmos endpoints; se for publicado, deve ser auditado separadamente (armazenamento do token, pinning, deep links).

Confiança resultante

- Código e API: **média-alta** — leitura integral dos controllers, validadores, middlewares, serviços e configuração; 15 testes locais.
- Frontend: **média** — sinks e armazenamento verificados; comportamento em produção (cabeçalhos, SW/PWA) não observado.
- Infraestrutura/operacional: **baixa** — não observável a partir do código.

Uma próxima etapa com maior confiança exigiria: (1) autorização para observação passiva do site publicado (`curl -I`, navegação sem exploração); (2) um ambiente de staging com credenciais de teste do Mercado Pago e conta admin de teste; (3) acesso somente leitura ao painel do Atlas/Vercel ou capturas de tela das configurações; (4) aprovação para instalar Semgrep/Gitleaks/OSV-Scanner em versões fixadas.

Apêndice A — Matriz de cobertura

co ntr ol_ id	domain	source_ standard	source_ require ment	ap pli ca ble	asset	test_ met hod	auth oriza tion_ need ed	status	eviden ce_ ref erence	finding_i d	limitation
V1 -0 1	V1 Encodin g and Sanitizati on	ASVS 5.0	Output encoding contextual no frontend (categoria; requisito exato não verificado)	sim	client React	revisão de código (grep sinks)	não	CONTROLE VALIDADO	AUD-CV-10		
V1 -0 2	V1 Encodin g and Sanitizati on	ASVS 5.0	Escape em templates de e-mail	sim	server mail.ts	revisão de código	não	CONTROLE VALIDADO	AUD-CV-17		
V1 -0 3	V1 Encodin g and Sanitizati on	ASVS 5.0	Escape de metacaracteres em consultas dinâmicas (regex)	sim	server controllers de listagem	teste local T05/T15	sim (cont a demo local)	FALHA	audit-eviden ce T05 T15	AUD-006	
V2 -0 1	V2 Validatio n and Business Logic	ASVS 5.0	Validação de esquema em toda entrada	sim	server validators (zod)	revisão de código	não	CONTROLE VALIDADO	AUD-CV-07		
V2 -0 2	V2 Validatio n and Business Logic	ASVS 5.0	Campos internos não atribuíveis pelo cliente (mass assignment)	sim	client.valid ators avatarUrl	teste local T08	sim	FALHA	audit-eviden ce T08	AUD-009	
V2 -0 3	V2 Validatio n and Business Logic	ASVS 5.0	Operações financeiras atômicas/idemp otentes	sim	billing.ts applyPay mentById	revisão de código	não	FALHA (PROVÁVEL)	billing.t s	AUD-007	sem credenciais do Mercado Pago para teste dinâmico
V2 -0 4	V2 Validatio n and Business Logic	ASVS 5.0	Limites de valores numéricos	sim	validators finance/g oal/client	revisão de código	não	CONTROLE VALIDADO	validat ors		
V3 -0 1	V3 Web Frontend Security	ASVS 5.0	Cabeçalhos de segurança no HTML (CSP frame-ancestors etc.)	sim	SPA na Vercel	revisão de configura ção (vercel.js on)	não	FALHA (PROVÁVEL)	vercel.j son	AUD-011	produção não observada

co ntr ol_ id	domain	source_sta ndard	source_require ment	ap pli ca ble	asset	test_met hod	auth oriza tion_ need ed	status	eviden ce_ref erence	finding_i d	limitation
V3 -0 2	V3 Web Frontend Security	ASVS 5.0	Cookies com HttpOnly/Secure /SameSite	sim	auth.contr oller setAuthCo okie	teste local T03 + código	sim	CONTROL E VALIDADO	audit- eviden ce T03	AUD-013	Secure só verificável em produção
V3 -0 3	V3 Web Frontend Security	ASVS 5.0	Proteção CSRF em defesa em profundidade	sim	API mutações	revisão de código	não	PARCIAL	auth.co ntroller /app.ts	AUD-013	
V3 -0 4	V3 Web Frontend Security	ASVS 5.0	Sem segredos/mapas de código no bundle	sim	client/dist	inspeção de build	não	CONTROL E VALIDADO	AUD- CV-10		
V3 -0 5	V3 Web Frontend Security	ASVS 5.0	Comportamento em logout/dispositiv o compartilhado	sim	client db clearLocal Data	revisão de código	não	PARCIAL	client/s rc/db/i ndex.ts	AUD-018	
V4 -0 1	V4 API and Web Service	ASVS 5.0	Autenticação em todos os endpoints de dados	sim	routes/ind ex.ts	teste local T08 (anônimo 401) + código	sim	CONTROL E VALIDADO	audit- eviden ce T08		
V4 -0 2	V4 API and Web Service	ASVS 5.0	CORS restritivo	sim	app.ts CORS	teste local T02/T02b	não	PARCIAL	audit- eviden ce T02	AUD-012	
V4 -0 3	V4 API and Web Service	ASVS 5.0	Webhooks assinados e limitados	sim	/api/billin g/webhoo k	teste local T11 + código	não	PARCIAL	audit- eviden ce T11	AUD-015	
V4 -0 4	V4 API and Web Service	ASVS 5.0	Limites de tamanho de requisição	sim	express.js on/multer	teste local T14	não	CONTROL E VALIDADO	audit- eviden ce T14	AUD-018	
V5 -0 1	V5 File Handling	ASVS 5.0	Validação de tipo por conteúdo e não por MIME declarado	sim	uploadAva tar.ts	revisão de código + sharp.for mat	não	FALHA (PROVÁVE L)	AUD- 002 evidên cia	AUD-002	exploit não executado
V5 -0 2	V5 File Handling	ASVS 5.0	Arquivos reprocessados/r enomeados e servidos com tipo correto	sim	utils/avata r.ts	revisão de código	não	CONTROL E VALIDADO	AUD- CV-16		
V5 -0 3	V5 File Handling	ASVS 5.0	Autorização em upload/remoção de arquivo	sim	client.cont roller uploadClie ntAvatar/d eleteClient	revisão de código	não	PARCIAL	client.c ontroll er	AUD-009	
V5 -0 4	V5 File Handling	ASVS 5.0	Path traversal na remoção local	sim	utils/avata r.ts deleteAvat ar	revisão de código	não	CONTROL E VALIDADO	path.b asena me		

control_id	domain	source_standard	source_requirement	aplicable	asset	test_method	authorization_needed	status	evidence_reference	finding_id	limitation
V6-01	V6 Authentication	ASVS 5.0	Sem credenciais padrão/embutidas	sim	seed.ts + banco produção	teste local T03 (login demo) + repo público	sim	FALHA	audit-evidencia T03; GitHub 200	AUD-001	
V6-02	V6 Authentication	ASVS 5.0	Política de senha (comprimento/blocklist)	sim	auth.validators	revisão de código	não	FALHA	auth.validators.ts	AUD-014	
V6-03	V6 Authentication	ASVS 5.0	Hash de senha adequado	sim	utils/password.ts	revisão de código + T13	não	CONTROLE VALIDADO	AUD-CV-04		
V6-04	V6 Authentication	ASVS 5.0	Anti-automação/rate limit por conta e IP real	sim	auth.routes.ts + app.ts	teste local T12	não	FALHA	audit-evidencia T12	AUD-003	
V6-05	V6 Authentication	ASVS 5.0	Recuperação de senha segura (token aleatório uso único expiração)	sim	services/verification.ts	revisão de código	não	CONTROLE VALIDADO	AUD-CV-05		
V6-06	V6 Authentication	ASVS 5.0	Link de recuperação com host confiável	sim	utils/appUrl.ts	revisão de código	não	FALHA (PROVÁVEL)	appUrl.ts	AUD-005	não testado envio real
V6-07	V6 Authentication	ASVS 5.0	Resposta/tempo uniforme (anti-enumeração)	sim	auth.controller	teste local T07/T13	não	FALHA	audit-evidencia T07 T13	AUD-010	
V6-08	V6 Authentication	ASVS 5.0	MFA para funções administrativas	sim	admin.controller	revisão de código	não	FALHA (design)	admin.controller.ts	AUD-008	
V6-09	V6 Authentication	ASVS 5.0	Verificação de e-mail antes de criar conta	sim	auth.controller.register	revisão de código + sessão anterior	não	CONTROLE VALIDADO	verification.ts		
V7-01	V7 Session Management	ASVS 5.0	Invalidação de sessão em logout/troca de senha	sim	jwt.ts/requireAuth.ts	revisão de código	não	FALHA	jwt.ts	AUD-004	
V7-02	V7 Session Management	ASVS 5.0	Expiração absoluta e por inatividade	sim	jwt.ts (7d)	revisão de código	não	PARCIAL	jwt.ts	AUD-004	
V7-03	V7 Session Management	ASVS 5.0	Identificador de sessão imprevisível	sim	JWT HS256 com segredo	revisão de código	não	CONTROLE VALIDADO	jwt.ts		

co ntr ol_ id	domain	source_sta ndard	source_require ment	ap pli ca ble	asset	test_met hod	auth oriza tion_ need ed	status	eviden ce_ref erence	finding_i d	limitation
V8-01	V8 Authorization	ASVS 5.0	Autorização por objeto (BOLA) em leitura/escrita/exclusão	sim	controllers de dados	teste local T08 + grep userId	sim	CONTROLE VALIDADO	AUD-CV-01		
V8-02	V8 Authorization	ASVS 5.0	Autorização de função (admin)	sim	admin.controller	teste local T09	sim	CONTROLE VALIDADO	AUD-CV-13		
V8-03	V8 Authorization	ASVS 5.0	Autorização por propriedade (campos internos)	sim	validators	teste local T08	sim	FALHA	audit-eviden ce T08	AUD-009	
V8-04	V8 Authorization	ASVS 5.0	Gate de entitlement (plano) no servidor	sim	requirePlans.ts	teste local T10	sim	CONTROLE VALIDADO	AUD-CV-14		
V9-01	V9 Self-contained Tokens	ASVS 5.0	Rejeição de alg none / algoritmos explícitos	sim	requireAuth.ts	teste local T04	não	PARCIAL	audit-eviden ce T04	AUD-004	
V9-02	V9 Self-contained Tokens	ASVS 5.0	Validação de iss/aud/exp	sim	jwt.ts	revisão de código	não	PARCIAL	jwt.ts	AUD-004	
V9-03	V9 Self-contained Tokens	ASVS 5.0	Revogação de tokens	sim	jwt.ts	revisão de código	não	FALHA	jwt.ts	AUD-004	
V10-01	V10 OAuth and OIDC	ASVS 5.0	—	não	—	—	—	NÃO APLICÁVEL			
V11-01	V11 Cryptography	ASVS 5.0	Aleatoriedade e hashing de tokens de verificação	sim	verification.ts	revisão de código	não	CONTROLE VALIDADO	AUD-CV-05		
V11-02	V11 Cryptography	ASVS 5.0	Segredo do JWT gerenciado por variável de ambiente e rotação	sim	env.ts	revisão de código	não	PARCIAL	env.ts	AUD-004	rotação não verificável
V12-01	V12 Secure Communication	ASVS 5.0	TLS/HSTS na borda	sim	Vercel	não testado (produção excluída)	sim	NÃO TESTADO		AUD-011	produção excluída
V12-02	V12 Secure Communication	ASVS 5.0	TLS para o banco e SMTP	sim	MONGO_URL ssl=true; SMTP 587 STARTTLS	revisão de configuração	não	CONTROLE VALIDADO	env/en v.example		

co ntr ol_ id	domain	source_sta ndard	source_require ment	ap pli ca ble	asset	test_met hod	auth oriza tion_ need ed	status	eviden ce_ref erence	finding_i d	limitation
V1 3- 01	V13 Configur ation	ASVS 5.0	Segredos fora do repositório	sim	git history	busca por padrões	não	CONTROL E VALIDADO	AUD- CV-09		
V1 3- 02	V13 Configur ation	ASVS 5.0	trust proxy correto atrás de proxy	sim	app.ts	teste local T12	não	FALHA	audit- eviden ce T12	AUD-003	
V1 3- 03	V13 Configur ation	ASVS 5.0	Configurações obrigatórias em produção (APP_URL MP_WEBHOOK_ SECRET)	sim	env.ts	revisão de código	não	FALHA	env.ts	AUD- 005;AUD- 015	
V1 3- 04	V13 Configur ation	ASVS 5.0	Debug desligado em produção	sim	app.ts morgan	revisão de código	não	CONTROL E VALIDADO	app.ts		
V1 4- 01	V14 Data Protectio n	ASVS 5.0	Minimização de dados em respostas	sim	toPublicUs er	revisão de código	não	PARCIAL	auth.co ntroller	AUD-018	
V1 4- 02	V14 Data Protectio n	ASVS 5.0	Backup e restauração	sim	MongoDB Atlas	não verificáve l	sim	NÃO TESTADO		AUD-018	sem acesso ao Atlas
V1 4- 03	V14 Data Protectio n	ASVS 5.0	Dados no cliente (IndexedDB) limpos ao sair	sim	client db	revisão de código	não	CONTROL E VALIDADO	client/s rc/db/i ndex.ts		
V1 5- 01	V15 Secure Coding and Architect ure	ASVS 5.0	Dependências atualizadas/lockf ile	sim	package- lock.json	npm audit	não	FALHA	npm audit	AUD- 016;AUD- 002	
V1 5- 02	V15 Secure Coding and Architect ure	ASVS 5.0	Tratamento de erros centralizado	sim	errorHand ler.ts	teste local T06	não	CONTROL E VALIDADO	AUD- CV-08		
V1 6- 01	V16 Security Logging and Error Handling	ASVS 5.0	Log de eventos de segurança	sim	server	revisão de código	não	FALHA	app.ts/ control lers	AUD-017	
V1 6- 02	V16 Security Logging and Error Handling	ASVS 5.0	Erros sem vazamento	sim	errorHand ler.ts	teste local T06	não	CONTROL E VALIDADO	AUD- CV-08		
V1 7- 01	V17 WebRTC	ASVS 5.0	—	nã o	—	—	—	NÃO APLICÁVEL			

co ntr ol_ id	domain	source_sta ndard	source_require ment	ap pli ca ble	asset	test_met hod	auth oriza tion_ need ed	status	eviden ce_ref erence	finding_i d	limitation
T1 0- A0 1	OWASP Top 10:2025	A01 Broken Access Control		sim	API	T08/T09/ T10	sim	CONTROLE VALIDADO (ressalvas AUD-009 AUD-013)	AUD- CV-01	AUD-009	
T1 0- A0 2	OWASP Top 10:2025	A02 Security Misconfigu ration		sim	Vercel/API	T01/T02/ T12	não	FALHA		AUD- 003;AUD- 011;AUD- 012;AUD- 015	
T1 0- A0 3	OWASP Top 10:2025	A03 Software Supply Chain Failures		sim	dependên cias	npm audit + npm ls	não	FALHA	npm audit	AUD- 002;AUD- 016	sem SBOM gerado
T1 0- A0 4	OWASP Top 10:2025	A04 Cryptogra phic Failures		sim	hash/JWT/ tokens	revisão de código	não	CONTROLE VALIDADO	AUD- CV- 04;AU D-CV- 05		
T1 0- A0 5	OWASP Top 10:2025	A05 Injection		sim	busca regex/NoS QL	T05 + código	sim	FALHA (regex)	audit- eviden ce T05	AUD-006	
T1 0- A0 6	OWASP Top 10:2025	A06 Insecure Design		sim	billing/ad min	revisão de código	não	FALHA	billing.t s	AUD- 007;AUD- 008	
T1 0- A0 7	OWASP Top 10:2025	A07 Authentica tion Failures		sim	auth	T03/T04/ T07/T12/ T13	sim	FALHA	audit- eviden ce	AUD- 001;AUD- 003;AUD- 004;AUD- 010;AUD- 014	
T1 0- A0 8	OWASP Top 10:2025	A08 Software or Data Integrity Failures		sim	build Vercel/CI	revisão (sem CI no repo)	não	NÃO TESTADO			pipeline da Vercel não observável
T1 0- A0 9	OWASP Top 10:2025	A09 Security Logging and Alerting Failures		sim	server	revisão de código	não	FALHA	app.ts	AUD-017	
T1 0- A1 0	OWASP Top 10:2025	A10 Mishandlin g of Exceptiona l Conditions		sim	errorHand ler/regex	T05/T06/ T14	não	PARCIAL	audit- eviden ce	AUD- 006;AUD- 018	

co ntr ol_ id	domain	source_sta ndard	source_require ment	ap pli ca ble	asset	test_met hod	auth oriza tion_ need ed	status	eviden ce_ref erence	finding_i d	limitation
AP I1	OWASP API Top 10:2023	API1 BOLA		sim	API	T08	sim	CONTROLE VALIDADO	AUD- CV-01		
AP I2	OWASP API Top 10:2023	API2 Broken Authentica tion		sim	auth	T03/T04/ T12/T13	sim	FALHA	audit- eviden ce	AUD- 001;AUD- 003;AUD- 004;AUD- 010	
AP I3	OWASP API Top 10:2023	API3 Broken Object Property Level Authorizati on		sim	client.valid ators	T08	sim	FALHA	audit- eviden ce T08	AUD-009	
AP I4	OWASP API Top 10:2023	API4 Unrestrict ed Resource Consumpti on		sim	busca/we bhook/rat e limit	T05/T11/ T12/T15	sim	FALHA	audit- eviden ce	AUD- 003;AUD- 006;AUD- 015	
AP I5	OWASP API Top 10:2023	API5 Broken Function Level Authorizati on		sim	admin	T09	sim	CONTROLE VALIDADO	AUD- CV-13		
AP I6	OWASP API Top 10:2023	API6 Unrestrict ed Access to Sensitive Business Flows		sim	billing	revisão de código	não	FALHA (PROVÁVEL)	billing.t s	AUD-007	
AP I7	OWASP API Top 10:2023	API7 SSRF		sim	mail/MP/B lob	revisão de código	não	CONTROLE VALIDADO	billing.t s/mail.t s		
AP I8	OWASP API Top 10:2023	API8 Security Misconfigu ration		sim	config	T01/T02/ T12	não	FALHA	audit- eviden ce	AUD- 003;AUD- 011;AUD- 012;AUD- 015	
AP I9	OWASP API Top 10:2023	API9 Improper Inventory Managem ent		sim	rotas	inventári o de rotas vs docs	não	CONTROLE VALIDADO	02- archite cture		
AP I10	OWASP API Top 10:2023	API10 Unsafe Consumpti on of APIs		sim	Mercado Pago/Rese nd/SMTP	revisão de código	não	PARCIAL	billing.t s	AUD- 007;AUD- 015	

Apêndice B — Glossário

BOLA / IDOR

O servidor recebe o identificador de um objeto mas não confirma que ele pertence a quem pediu.

CSRF

Um site externo faz o navegador da vítima enviar uma requisição usando a sessão dela.

Mass assignment

O cliente consegue gravar campos que só o servidor deveria definir.

ReDoS

Expressão regular construída para consumir tempo exponencial e travar o processamento.

Rate limiting

Limite de quantas vezes uma ação pode ser repetida em uma janela de tempo.

JWT

Token assinado que carrega a identidade do usuário; o servidor confia na assinatura em vez de consultar uma tabela de sessões.

Race condition / TOCTOU

Duas execuções simultâneas leem o mesmo estado antes que qualquer uma o altere, produzindo efeito duplicado.

CVSS v4.0

Escala técnica de 0 a 10 para gravidade; não substitui o contexto de negócio.

CONFIRMADO / PROVÁVEL / HIPÓTESE

Evidência reproduzível / evidência parcial com elo faltante explicado / sem evidência direta.

Apêndice C — Referências

- OWASP ASVS 5.0 — <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP Top 10:2025 — <https://owasp.org/Top10/2025/>
- OWASP API Security Top 10:2023 — <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- NIST SP 800-63B-4 — <https://csrc.nist.gov/pubs/sp/800/63/b/4/final>
- RFC 8725 JWT BCP — <https://www.rfc-editor.org/info/rfc8725/>
- FIRST CVSS v4.0 — <https://www.first.org/cvss/v4.0/specification-document>
- Avisos GitHub citados: GHSA-rgj7-g3m4-5g8c (sharp), GHSA-wc9g-mqfw-jrwm / GHSA-qfvm-cv95-jqjf / GHSA-535w-7cp7-47q4 (multer)

Apêndice D — Evidência bruta dos testes locais

```
### T01 security headers on API (helmet)
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action
'self';frame-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr
'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=15552000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN

### T02 CORS: foreign origin (expect no ACA0)

### T02b CORS: http://localhost origin always allowed with credentials
Access-Control-Allow-Origin: http://localhost
Access-Control-Allow-Credentials: true

### T03 login sets cookie (flags)
Set-Cookie: token=[REDACTED]; Max-Age=604800; Path=/; Expires=Mon, 21 Sep 2026 00:25:56 GMT; HttpOnly;
SameSite=Lax

### T04 JWT alg=none rejected?
status: 401

### T05 regex injection in search (expect 500 if unescaped)
search='(' -> 500
search='.*' matches everything of own user -> 4 clients

### T06 unknown route / malformed JSON (error leakage)
unknown: {"message":"Rota não encontrada."}
malformed: {"message":"Corpo da requisição inválido."}

### T07 user enumeration via register/request-code (existing e-mail, no mail sent)
{"message":"Este e-mail já está cadastrado. Entre com sua senha ou use \"Esqueci minha senha\"."}

### T08 BOLA / mass assignment with canary user
demo creates canary client: 201
canary user GET demo's client -> 404
canary user PUT demo's client -> 404
canary user DELETE demo's client -> 404
canary user list clients count -> 0
anonymous GET client -> 401
demo sets avatarUrl to external URL (mass assignment) -> stored avatarUrl =
https://attacker.example/track.png
demo deletes canary client -> 204

### T09 admin endpoint as normal user
status: 403

### T10 plan gate (canary expired -> 402)
(expire canary, then) clients -> 200 (run after expire step)

### T11 webhook without signature secret accepts any body?
status: 200
```

```
### T12 rate limiter keyed by req.ip (X-Forwarded-For ignored: trust proxy off)
22 failed logins, each with a DIFFERENT X-Forwarded-For: 401 401 401 401 401 401 401 401 401 401 401 401
401 401 401 401 401 401 429 429 429 429
(429 appearing means all were counted in ONE bucket -> behind a proxy every visitor shares the same
limit)
T10 canary (expired) GET /clients -> 402
T10b canary (expired) GET /billing/status -> False expired
### T13 timing difference: unknown e-mail vs existing e-mail with wrong password (ms)
unknown=0.108209 existing=0.359680
unknown=0.017486 existing=0.286373
unknown=0.018846 existing=0.285995
### T14 express.json body limit (default 100kb) -> 500
### T15 search length: 5000-char regex accepted? -> 200
```